Security Review

Cantina Managed

SOLANA: PRIVATE CHANNELS RE-REVIEW

Cantina Managed review by:
R0bert, Lead Security Researcher
Sujith S, Lead Security Researcher

CANTINA

October 19, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3
- 2.2 Cantina Managed Team Statement . . . 3

3 Findings **4**
- 3.1 High Risk . . . 4
  - 3.1.1 Escrow resync reopens reminted withdrawals . . . 4
  - 3.1.2 Forged memos suppress credits after resync . . . 4
  - 3.1.3 Admin rotation reopens serviced deposits . . . 5
  - 3.1.4 Fabricated wSOL buys backed tokens . . . 6
  - 3.1.5 Refunded withdrawals drain the release payer . . . 6
  - 3.1.6 Dirty cache pressure erases receipt balances . . . 7
  - 3.1.7 Retired mint authority can drain escrow . . . 8
- 3.2 Medium Risk . . . 9
  - 3.2.1 Withdrawals settle before Solana finality . . . 9
  - 3.2.2 Resync can replay in flight bridge effects . . . 9
  - 3.2.3 Stalled reads exhaust gateway connections . . . 10
  - 3.2.4 Malformed or ambiguous chain data is silently checkpointed . . . 10
  - 3.2.5 The reconciliation halt fails open and is not atomic with transaction claims . . . 11
  - 3.2.6 Full-width token releases are truncated and create phantom liabilities . . . 12
  - 3.2.7 Batched AllowMint instructions can use another mint's event metadata . . . 13
  - 3.2.8 Unthrottled wallet challenges enable persistent database-write denial of service . . . 14
  - 3.2.9 Tiny withdrawals drain the release payer . . . 15
  - 3.2.10 Gasless transfers create permanent accounts . . . 15
  - 3.2.11 Hook errors permanently block bitmap rotation . . . 16
  - 3.2.12 Hook resolution stalls every withdrawal . . . 17
  - 3.2.13 Memo upgrade blocks historical release replay . . . 17
  - 3.2.14 Mint reallow leaves stale withdrawal data . . . 18
  - 3.2.15 Same slot blocks suppress valid deposits . . . 19
  - 3.2.16 V1 upgrade rejects prior transaction rows . . . 19
  - 3.2.17 Token account closures can squat future mints . . . 20
  - 3.2.18 Async address index reopens serviced deposits . . . 21
  - 3.2.19 Bulk block queries overwhelm ordinary reads . . . 21
  - 3.2.20 Expired mint transactions can execute twice . . . 22
  - 3.2.21 Gap repair can regress mint metadata . . . 23
  - 3.2.22 Gap repair can strand authorized deposits . . . 24
  - 3.2.23 Native SOL deposits remain locked in escrow . . . 25
  - 3.2.24 Read nodes report live blockhashes expired . . . 25
  - 3.2.25 Inconsistent finality reads can duplicate mints . . . 26
  - 3.2.26 Replica skew leaks data after account handoffs . . . 27
  - 3.2.27 Slow webhooks corrupt recovered transactions . . . 28
  - 3.2.28 Stale state reads strand reopened withdrawals . . . 29
  - 3.2.29 Stale snapshots can trigger a global halt . . . 30
  - 3.2.30 Unapproved mints block escrow indexer startup . . . 31
  - 3.2.31 Absent history cursors scan every address row . . . 32
  - 3.2.32 Failed startup fill strands withdrawals . . . 32
  - 3.2.33 Pings prevent Yellowstone stream recovery . . . 33
  - 3.2.34 Quiet slots strand withdrawal refunds . . . 33
  - 3.2.35 Stalled RPC response stops withdrawal indexing . . . 34
  - 3.2.36 Zero-value deposits create operator-funded channel state . . . 34
  - 3.2.37 Operational scripts expose secrets in process arguments . . . 35
  - 3.2.38 Admin env helper creates world-readable key files . . . 35
  - 3.2.39 Transfer fees can erase withdrawal payouts . . . 36

3.2.40 Unavailable bitmap validation lets the withdrawal operator start unsafely 37
3.2.41 CPI deposits without stack heights permanently block replay 37
3.3 Low Risk 38
3.3.1 Fee monitor watches an unrelated wallet 38
3.3.2 Privy signer configuration panics at startup 39
3.3.3 Storage errors produce a fake blockhash 39
3.3.4 Unused writable keys serialize channel execution 39
3.3.5 Zero allowance delegates can read private state 40
3.3.6 Full resync can exhaust indexer memory 41
3.3.7 Withdraw startup permits unusable configuration 42
3.3.8 Parsed RPC encoding halts block ingestion 42
3.3.9 Transient journal read parks paid withdrawal 43
3.3.10 Unsafe connection caps defeat IP isolation 43
3.3.11 Whitespace JWT secret disables gateway RBAC 44
3.3.12 Zero backfill batch size exhausts the indexer 45
3.3.13 Zero retry budget quarantines withdrawals 45
3.3.14 Zero verifier count stops the write node 46
3.3.15 Reconciliation script can certify an insolvent escrow 46
3.3.16 Zero operator batch size stalls all bridge work 47
3.3.17 Missing webhook suppresses insolvency alerts 48
3.3.18 Invalid operator signer silently uses admin 48
3.3.19 Weak JWT secrets permit session forgery 49
3.3.20 Invalid mint options initialize private mints 49
3.3.21 Startup supply check ignores bps tolerance 50
3.3.22 Zero auth database pool crashes both services 51
3.3.23 Zero sample period stops the write node 51
3.3.24 Zero channel capacity crashes the operator 52
3.3.25 Zero connection limit disables core RPC 52
3.3.26 Username throttling does not limit credential guesses 53
3.3.27 Unbounded pending-remint recovery can stall withdrawal startup 53
3.3.28 Invalid CORS configuration crashes gateway requests 54
3.3.29 Zero retention silently deletes every successful backup 55
3.3.30 Zero operator intervals hammer dependencies and defeat reconciliation debounce 55
3.3.31 Critical task exits panic operator shutdown 56
3.3.32 Concurrent schema setup aborts an indexer 56
3.3.33 Yellowstone builds require the RPC feature 57

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

The Solana Foundation is a non-profit foundation based in Zug, Switzerland, dedicated to the decentralization, adoption, and security of the Solana ecosystem.

From Sep 21st to Oct 9th the Cantina team conducted a review of solana-private-channels on commit hash 641a5f1b. The team identified a total of **81** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	7	7	0
Medium Risk	41	39	2
Low Risk	33	33	0
Gas Optimizations	0	0	0
Informational	0	0	0
Total	**81**	**79**	**2**

2.1 Scope

The security review had the following components in scope for solana-private-channels on commit hash 641a5f1b:

```
├── auth/
│   └── src/
│       ├── bin/
│       │   └── admin.rs
│       ├── routes/
│       │   ├── challenge.rs
│       │   ├── login.rs
│       │   ├── mod.rs
│       │   ├── register.rs
│       │   ├── verify_wallet.rs
│       │   └── wallets.rs
│       ├── config.rs
│       ├── db.rs
│       ├── error.rs
│       ├── jwt.rs
│       ├── lib.rs
│       ├── main.rs
│       ├── models.rs
│       ├── password.rs
│       ├── pool_status.rs
│       ├── serve.rs
│       ├── throttle.rs
│       └── validation.rs
├── core/
│   └── src/
│       ├── accounts/
│       │   ├── address_index_repair.rs
│       │   ├── bob.rs
│       │   ├── get_signatures_for_address.rs
│       │   ├── get_transaction.rs
│       │   ├── postgres.rs
│       │   ├── redis.rs
│       │   ├── traits.rs
│       │   ├── types.rs
```

```
│       │   └── write_batch.rs
│       ├── rpc/
│       │   ├── api.rs
│       │   ├── constants.rs
│       │   ├── decode.rs
│       │   ├── error.rs
│       │   ├── get_account_info_impl.rs
│       │   ├── get_block_height_impl.rs
│       │   ├── get_block_impl.rs
│       │   ├── get_block_time_impl.rs
│       │   ├── get_blocks_impl.rs
│       │   ├── get_blocks_with_limit_impl.rs
│       │   ├── get_epoch_info_impl.rs
│       │   ├── get_epoch_schedule_impl.rs
│       │   ├── get_first_available_block_impl.rs
│       │   ├── get_latest_blockhash_impl.rs
│       │   ├── get_recent_blockhash_impl.rs
│       │   ├── get_recent_performance_samples_impl.rs
│       │   ├── get_signature_statuses_impl.rs
│       │   ├── get_signatures_for_address_impl.rs
│       │   ├── get_slot_impl.rs
│       │   ├── get_slot_leaders_impl.rs
│       │   ├── get_supply_impl.rs
│       │   ├── get_token_account_balance_impl.rs
│       │   ├── get_transaction_count_impl.rs
│       │   ├── get_transaction_impl.rs
│       │   ├── get_vote_accounts_impl.rs
│       │   ├── handler.rs
│       │   ├── is_blockhash_valid_impl.rs
│       │   ├── mod.rs
│       │   ├── rpc_impl.rs
│       │   ├── send_transaction_impl.rs
│       │   ├── server.rs
│       │   └── simulate_transaction_impl.rs
│       ├── stages/
│       │   ├── address_index_writer.rs
│       │   ├── dedup.rs
│       │   ├── execution.rs
│       │   ├── mod.rs
│       │   ├── sequencer.rs
│       │   ├── settle.rs
│       │   └── sigverify.rs
│       ├── vm/
│       │   ├── admin.rs
│       │   ├── clock.rs
│       │   ├── gasless_callback.rs
│       │   └── mod.rs
│       └── processor.rs
├── gateway/
│   └── src/
│       ├── bin/
│       │   └── gateway.rs
│       ├── auth.rs
│       ├── db.rs
│       ├── lib.rs
│       └── metrics.rs
├── indexer/
│   └── src/
│       ├── indexer/
│       │   ├── datasource/
│       │   │   ├── common/
```

```
│       │   │   │   ├── parser/
│       │   │   │   │   ├── escrow.rs
│       │   │   │   │   ├── mod.rs
│       │   │   │   │   ├── pubkey.rs
│       │   │   │   │   └── withdraw.rs
│       │   │   │   ├── datasource.rs
│       │   │   │   ├── mod.rs
│       │   │   │   └── types.rs
│       │   │   ├── rpc_polling/
│       │   │   │   ├── decoder.rs
│       │   │   │   ├── mod.rs
│       │   │   │   ├── rpc.rs
│       │   │   │   ├── source.rs
│       │   │   │   └── types.rs
│       │   │   ├── yellowstone/
│       │   │   │   ├── convert.rs
│       │   │   │   ├── mod.rs
│       │   │   │   └── source.rs
│       │   │   └── mod.rs
│       │   ├── backfill.rs
│       │   ├── checkpoint.rs
│       │   ├── indexer.rs
│       │   ├── mod.rs
│       │   ├── reconciliation.rs
│       │   ├── resync.rs
│       │   └── transaction_processor.rs
│       └── operator/
│           ├── sender/
│           │   ├── mint.rs
│           │   ├── mod.rs
│           │   ├── proof.rs
│           │   ├── remint.rs
│           │   ├── state.rs
│           │   ├── transaction.rs
│           │   └── types.rs
│           ├── utils/
│           │   ├── account_util.rs
│           │   ├── instruction_util.rs
│           │   ├── mint_util.rs
│           │   ├── mod.rs
│           │   ├── rpc_util.rs
│           │   ├── signer_util.rs
│           │   ├── storage_util.rs
│           │   └── transaction_util.rs
│           ├── constants.rs
│           ├── db_transaction_writer.rs
│           ├── escrow_sweep.rs
│           ├── feepayer_monitor.rs
│           ├── fetcher.rs
│           ├── mod.rs
│           ├── operator.rs
│           ├── processor.rs
│           ├── reconciliation.rs
│           └── recovery.rs
├── private-channel-escrow-program/
│   └── program/
│       └── src/
│           ├── processor/
│           │   ├── shared/
│           │   │   ├── account_check.rs
│           │   │   ├── event_utils.rs
```

```
│           │   │   ├── mod.rs
│           │   │   ├── pda_utils.rs
│           │   │   ├── token_utils.rs
│           │   │   └── utils.rs
│           │   ├── add_operator.rs
│           │   ├── allow_mint.rs
│           │   ├── block_mint.rs
│           │   ├── create_instance.rs
│           │   ├── deposit.rs
│           │   ├── mod.rs
│           │   ├── process_emit_event.rs
│           │   ├── release_funds.rs
│           │   ├── remove_operator.rs
│           │   ├── rotate_bitmap.rs
│           │   └── set_new_admin.rs
│           ├── state/
│           │   ├── allowed_mint.rs
│           │   ├── discriminator.rs
│           │   ├── instance.rs
│           │   ├── mod.rs
│           │   ├── operator.rs
│           │   └── withdrawal_bitmap.rs
│           ├── constants.rs
│           ├── entrypoint.rs
│           ├── error.rs
│           ├── events.rs
│           ├── instructions.rs
│           └── lib.rs
├── private-channel-withdraw-program/
│   └── program/
│       └── src/
│           ├── processor/
│           │   ├── shared/
│           │   │   ├── account_check.rs
│           │   │   ├── mod.rs
│           │   │   ├── token_utils.rs
│           │   │   └── utils.rs
│           │   ├── mod.rs
│           │   └── withdraw_funds.rs
│           ├── discriminator.rs
│           ├── entrypoint.rs
│           ├── error.rs
│           ├── events.rs
│           ├── instructions.rs
│           └── lib.rs
└── scripts/
    └── devnet/
        └── src/
            ├── bin/
            │   ├── add_operator.rs
            │   ├── allow_mint.rs
            │   ├── block_mint.rs
            │   ├── create_instance.rs
            │   ├── deposit.rs
            │   └── withdraw.rs
            └── lib.rs
```

2.2 Cantina Managed Team Statement

Review depth. The review was conducted on a best-effort basis within a fixed time allocation. The in-scope codebase comprises 173 Rust files totalling approximately 32,000 normalized source lines

of code (nSLOC), or approximately 25,600 after weighting for logic complexity, covering on-chain programs as well as the off-chain components. Two Lead Security Researchers reviewed it over 18 days (Sep 21st to Oct 9th). Given the size and complexity of the codebase, this allocation did not allow for exhaustive coverage of every component. The review is therefore not intended to be comprehensive, and vulnerabilities may remain that were not identified. All findings reported were resolved by the project team. Cantina recommends an additional review of the final commit, with particular attention to the components that yielded the most findings, prior to production deployment.

3 Findings

3.1 High Risk

3.1.1 Escrow resync reopens reminted withdrawals

Severity: High Risk.

Context: decoder.rs#L106-L146, indexer.rs#L308-L326, resync.rs#L137-L148, resync.rs#L332-L407, transaction_processor.rs#L126-L152, transaction_processor.rs#L508-L630, fetcher.rs#L74-L96, processor.rs#L907-L976, db.rs#L171-L185.

Description: The escrow and withdrawal roles share the `transactions` table, but a resync rebuilds only the program selected by its configuration. Every resync first drops the complete shared schema including all withdrawal rows and remint signatures. It also builds a `ConsumedSet` containing private channel remints and attaches that set to the resync transaction processor. However, an escrow resync configures its backfill for `ProgramType::Escrow`, whose decoder emits only escrow program instructions. The consumed remint entries therefore have no withdrawal rows to reconcile before the resync exits. The safety check for an advanced withdrawal bitmap returns early for any program other than `ProgramType::Withdraw`, so it does not block this escrow resync. The rebuilt observed release table cannot protect such a row because a reminted withdrawal has no successful release to observe.

When the ordinary withdrawal indexer restarts against the rebuilt database, its transaction processor is created without the consumed set. It backfills the historical `WithdrawFunds` instruction as a new `Pending` withdrawal. The withdrawal operator fetches that row and forwards a new `ReleaseFunds` transaction without first checking whether the source event already has an exact finalized remint on the private channel.

An attacker can prepare the condition by withdrawing to a destination whose associated token account does not yet exist. The Solana release fails, so the operator compensates the burn with a private channel remint. The attacker can then create the destination account. If an administrator subsequently runs the supported escrow resync, the historical withdrawal becomes pending again. Provided its restored nonce is still in the current bitmap generation and clear, the new release succeeds. The attacker retains the reminted private tokens and also receives the escrowed Solana tokens. Each replay reduces custody without reducing private channel supply. Multiple reminted withdrawals can be released before reconciliation halts the operators.

Recommendation: Make a destructive resync restore the complete shared database under one exclusive lock. Backfill both escrow and withdrawal histories with the same immutable consumed set captured before the drop. Do not release the lock until both checkpoints and every terminal remint verdict are durable. Alternatively, make the destructive operation specific to one role and preserve the other role's transaction rows, nonce sequence, attempt journals and observed releases.

As defense in depth, gate every rebuilt withdrawal before release against an exact finalized remint transaction for its source event ID, mint, recipient, authority and amount. Do not accept a memo alone as proof because public callers can forge the idempotency memo.

Solana Foundation: Fixed in commit 4dbe498.

Cantina: Fix verified. `ResyncService::run_resync` uses the role-scoped `wipe_program_fenced`, and `PostgresDb::wipe_program_on` deletes only the selected program's rows. An escrow resync therefore preserves withdrawal rows, including terminal remints and their signatures, so it cannot rebuild an already compensated withdrawal as pending.

3.1.2 Forged memos suppress credits after resync

Severity: High Risk.

Context: mint.rs#L435-L480.

Description: A resync rebuilds source events and uses `ConsumedSet` to keep deposits and withdrawal remints that already executed on the private channel in a terminal state. `enumerate_consumed_mints()` builds this set from `getSignaturesForAddress(authority)`. It accepts every successful history

entry whose memo starts with `private_channel:mint-idempotency:` or `private_channel:remint:`. It does not fetch the transaction, require the authority to be a signer or verify that the transaction contains the expected `MintTo`.

An address history entry does not prove authority. The core indexes a transaction under every message key, including an unused read only key. The public `sendTransaction` method also admits System transfers and Memo instructions. A caller can therefore observe a finalized source event, compute its deterministic `SourceEventId` and submit a valid transaction signed only by the caller. The transaction carries the forged memo and lists the configured authority as an unused non signer key.

If a deposit has not been minted when an administrator later runs the supported resync operation, the forged entry is added to `ConsumedSet`. `reconcile_against_consumed()` rebuilds the deposit as `Completed` and records the attacker's transaction as its counterpart. The operator will not mint the private tokens even though the user's source tokens remain in escrow. The remint prefix has the same effect on a failed withdrawal. It rebuilds the row as `FailedReminted` although the user received neither a Solana release nor private channel compensation. A caller can persist markers for every observed source event and the amount or number of affected users is not bounded.

Recommendation: Fetch and authenticate every transaction that contributes an entry to `ConsumedSet`. Require the transaction to have succeeded, contain the configured authority as a signer, contain the exact idempotency memo and execute the expected `MintTo` for the rebuilt event's mint, recipient and amount. Abort the resync if a prefixed marker cannot be authenticated.

Solana Foundation: Fixed in commit 5fcd732.

Cantina: Fix verified. Resync enumeration accepts a deposit marker only when the transaction succeeded, the expected authority signed, the marker is exact, and exactly one authority `MintTo` matches it. The full candidate is revalidated before destructive state replacement. An arbitrary memo can no longer mark a deposit as already credited.

3.1.3 Admin rotation reopens serviced deposits

Severity: High Risk.

Context: indexer.rs#L563-L579, resync.rs#L100-L134, transaction_processor.rs#L126-L152, mint.rs#L429-L445.

Description: The destructive resync is designed to rebuild every deposit already minted on the private channel as terminal. `run_resync()` takes the current `ADMIN_SIGNER` public key and `build_consumed_set()` passes only that key to `enumerate_consumed_mints()`. The enumeration then treats the complete result of `getSignaturesForAddress` for this one address as the complete history of serviced deposits.

The documented channel admin rotation moves receipt mint authority to a new key with SPL `Set Authority`. This cannot add the new key to mint transactions that the old authority signed. As a result, a resync after the rotation receives a successful but incomplete history that omits every earlier mint transaction unless the new key happened to be another account in those messages.

The resync subsequently drops the transaction and attempt tables. When source history is rebuilt, `reconcile_against_consumed()` leaves each omitted deposit as `Pending`. The old mint attempt is no longer available to `gate_reopened_deposit()` and the operator submits another `MintTo` under the new authority.

An authorized resync whose source range crosses an admin rotation can therefore duplicate every deposit serviced by the prior authority. Each duplicate increases private channel supply without adding tokens to escrow, which can create a material custody deficit.

Recommendation: Persist a complete append only lineage of channel mint authorities across rotations. Require resync to enumerate and union the history of every authority in that lineage before deleting any state. Refuse the operation when the lineage is unavailable or incomplete.

Solana Foundation: Fixed in commit 543ab84.

Cantina: Fix verified. Resync reconstructs consumed deposits from every relevant receipt-mint authority history rather than only the current administrator. Each historical authority/mint relation is validated and the completed floor is preserved, so rotating admin cannot hide prior service and reopen deposits.

3.1.4 Fabricated wSOL buys backed tokens

Severity: High Risk.

Context: execution.rs#L797-L890, admin.rs#L77-L115, admin.rs#L335-L362, gasless_callback.rs#L81-L99.

Description: The gasless executor gives an unknown fee payer ten temporary lamports so the SVM can validate and execute its transaction. These lamports are not backed by a deposit. `enforce_lamport_conservation()` is therefore intended to stop them from becoming persistent user value.

The check permits one missing fabricated lamport for every account created by the transaction. It then clears only the fabricated fee payer and leaves each created account with its one lamport existence balance. A user can create an associated token account with this subsidy, close it to a persistent System account in a later transaction and repeat the cycle. The close transaction does not spend its own fabricated fee payer balance, so the check treats the credited destination as a transfer of existing value. Each cycle consequently adds another persistent lamport that no user deposited.

The private runtime uses free rent and `AdminVm` can initialize the native mint address without a special restriction. The user can move the accumulated lamports into a native mint token account and call `SyncNative`, which converts the complete balance into transferable wSOL. The supported DvP program accepts this native mint as a payment leg. A user can therefore exchange the fabricated wSOL for a counterparty's bridge backed tokens. Those acquired tokens pass the withdrawal program's normal burn and release flow, so their withdrawal removes real assets from Solana escrow.

This path requires the private native mint to be initialized and a counterparty to fund a DvP trade that accepts private channel wSOL. Both the native mint and DvP are documented channel flows. The fabrication is repeatable with fresh fee payers and charges no channel fee, so the amount of backed assets obtained and withdrawn is not bounded by the attacker's deposited value.

Recommendation: Track the one lamport creation subsidy across transactions and consume it when the subsidized account closes instead of crediting it to the close destination. Only lamports proven to originate from a persistent funded account should survive an account closure or contribute to native token balance. Reject initialization and use of the native mint until this provenance rule is active so a synthetic account floor cannot become a transferable payment asset.

Solana Foundation: Fixed in commit e76ac25.

Cantina: Fix verified. The native SOL mint is rejected consistently at channel send/simulate validation, in `AdminVM`, and by the escrow program's `AllowMint` processor. Neither an administrator nor a transaction path can create a private representation of fabricated wSOL.

3.1.5 Refunded withdrawals drain the release payer

Severity: High Risk.

Context: processor.rs#L486-L567, processor.rs#L743-L808, processor.rs#L925-L976, remint.rs#L273-L345, remint.rs#L781-L848, transaction.rs#L907-L913, transaction.rs#L1517-L1663, transaction_util.rs#L120-L170, transaction_util.rs#L217-L239, token_utils.rs#L138-L175, withdraw_funds.rs#L30-L66.

Description: A failed Solana release should not let one private token balance consume the bridge's settlement budget repeatedly. However, the withdrawal program accepts any nonzero amount and a user selected Solana destination, then burns the amount without reserving a nonrefundable settlement fee. The escrow also accepts Token-2022 mints with every extension, including `ConfidentialTransferMint`.

The owner of a Token-2022 Confidential Transfer account can configure its associated token account and invoke the public `DisableNonConfidentialCredits` instruction. The operator derives that account from the withdrawal's user selected destination. Its withdrawal preflight checks the pooled escrow account for freeze state and checks the mint for pause and permanent delegate risks, but it never checks whether the destination account refuses a normal transfer.

The operator therefore signs and broadcasts `ReleaseFunds` with the admin as the Solana fee payer. Token-2022 rejects the inner `TransferChecked` with `NonConfidentialTransfersDisabled`. Solana atomically rolls back the escrow balance and withdrawal nonce, but still charges the admin's transaction fee. The confirmation parser recognizes only escrow custom errors 11 through 14, so Token-2022 custom error 49 becomes a generic permanent failure.

The sender then waits for finality and proves that every release signature failed and that no release record exists. It then remints the exact burned amount to the initiator. The user can submit the same withdrawal to the same rejecting destination after each remint. No asset ever leaves escrow and the attacking balance is never consumed, but every iteration spends the shared payer's SOL. Repetition can deplete that payer and make all legitimate withdrawals unavailable until it is replenished. A minimum withdrawal amount alone does not bound this loop because the full amount is restored after every failed release.

Recommendation: Reserve a nonrefundable user funded settlement fee before accepting the private burn and exclude that fee from every compensating remint. Configure each mint's minimum withdrawal so the principal and settlement reserve are both representable and reject a withdrawal that cannot cover them. The operator can additionally park a destination that already disables nonconfidential credits, but this mutable preflight should remain defense in depth rather than the economic boundary.

Solana Foundation: Fixed in commit ddf1129.

Cantina: Fix verified. Each mint can charge a configured release fee to the treasury before the user's private balance is burned. If release fails, remint restores only the burned principal, not a second fee. This aligns retry/refund accounting and prevents repeated failed withdrawals from consuming the payer without compensation.

3.1.6 Dirty cache pressure erases receipt balances

Severity: High Risk.

Context: bob.rs#L243-L280, bob.rs#L327-L375, bob.rs#L479-L553.

Description: `BOB::preload_accounts()` is responsible for making every transaction account available to the SVM from either its cache or PostgreSQL. It first classifies each requested key as a cache hit or a miss. It then calls `garbage_collect()`, whose `cap_evict()` pass removes clean entries when the cache contains more than one million entries. This entry cap does not protect the keys in the current preload.

An entry that was classified as a hit can therefore be removed before execution. The key is absent from the previously computed `miss_keys` list, so the function does not reload it from PostgreSQL. The account loader consequently reports that durable account as nonexistent to the SVM.

An unauthenticated caller can create the required dirty population through the public transaction endpoint. An SPL Token `AmountToUiAmount` instruction reads its mint but accepts ignored trailing account metas. A v1 transaction can pass 61 unique trailing accounts as writable while keeping the shared mint read only. The transaction executes successfully and `BOB::update_accounts()` marks every writable output dirty even though the instruction did not modify those accounts. The zero lamport outputs also consume no account data byte budget. Sustained traffic can place more than one million such entries ahead of settlement because the executor to settler queue can retain up to 1,000 result messages and row based backpressure begins below that queue.

The caller can then keep a chosen receipt mint dirty and make a source deposit whose recipient is the victim. The operator handles that deposit with an idempotent associated token account creation followed by `MintTo`. During preload, the dirty population cannot be evicted, so `cap_evict()` removes the victim's clean token account instead. The associated token account program sees an absent account and recreates it with a zero balance before `MintTo` adds only the new deposit amount. For example, an existing balance of 100 followed by a deposit of 10 becomes 10 instead of 110.

This overwrites the prior token account row and permanently destroys the victim's receipt balance. The receipt mint supply is not reduced by the overwrite, so the existing supply against custody reconciliation still remains balanced after the new deposit. Exploitation requires the caller to keep execution at least one million writable account outputs ahead of settlement. The public gateway's per IP rate limit raises the traffic cost but does not impose a global limit.

Recommendation: Run `garbage_collect()` before deriving cache hits and `miss_keys` in `preload_accounts()`. This ensures that every key evicted by the entry cap is classified as a miss and reloaded from PostgreSQL before execution. Do not run any later entry eviction pass unless it protects the complete current preload set or adds every removed hit back to the miss list.

Solana Foundation: Fixed in commit 862375a.

Cantina: Fix verified. Cache eviction now occurs after hit/miss loading and protects the full set being preloaded. Entry-count and byte caps are applied consistently, so dirty receipt accounts needed for the batch cannot be evicted and later overwritten with stale balances.

3.1.7 Retired mint authority can drain escrow

Severity: High Risk.

Context: sigverify.rs#L113-L137, transactions.rs#L6-L28, escrow_sweep.rs#L235-L260, reconciliation.rs#L264-L314, reconciliation.rs#L490-L515.

Description: The private channel must stop accepting token issuance from an administrator after the deployment replaces that key. Receipt mints instead store their authority in each SPL Mint account independently of the administrator keys configured on the node.

`ADMIN_INSTRUCTIONS_MAP` classifies only `InitializeMint` and `InitializeMint2` as administrative SPL Token instructions. `MintTo` remains a normal instruction. `sigverify_transaction()` requires a configured administrator signature only for an administrative transaction. It verifies the transaction signatures without checking the configured administrator set for a normal transaction. As a result, a retired key that remains in a receipt mint's `mint_authority` field can still submit an arbitrary `MintTo` after the node recognizes only its replacement.

Runtime reconciliation does not detect the dangerous authority state. `fetch_channel_supply()` unpacks the mint but returns only `Mint.supply`. `load_halt_inputs()` therefore compares supply with custody and never checks `mint_authority` or `freeze_authority`. The retired authority can combine `MintTo` with an attacker signed `WithdrawFunds` in one transaction. The first instruction creates an unbacked balance and the second burns it. Mint supply has the same value before and after the transaction, so a periodic reconciliation read cannot observe the temporary excess.

The withdrawal program accepts the resulting balance and emits the ordinary withdrawal data. The indexer stores it as a pending withdrawal and the registered operator builds `ReleaseFunds` for the recorded mint, amount and attacker selected destination. The source escrow program has no information about the private mint authority, so it accepts the operator signed release and transfers real custody to the attacker.

This affects a rotation that updates the configured administrator before every initialized receipt mint authority has moved to the replacement key. The repository documentation instructs operators to migrate each mint with `SetAuthority`, but the bridge does not fail closed when that manual step is omitted or incomplete. A compromised retired key can choose any amount available in custody and repeat the withdrawal across every mint that still names it. Honest receipt holders are left with undercollateralized balances after the release.

Recommendation: Treat the receipt mint authority as a fail closed bridge invariant. Before either bridge pipeline starts and during each reconciliation tick, load the complete private mint state for every supported source mint. Require `mint_authority` to equal the configured channel administrator and require any present `freeze_authority` to equal the same key. Persist the reconciliation halt before any further deposit or withdrawal work when a mismatch appears. Keep both pipelines halted during authority migration and resume only after every initialized mint has been verified under the replacement key.

Solana Foundation: Fixed in commit 7586afe.

Cantina: Fix verified. `requires_admin_signer` now covers `MintTo`, `MintToChecked`, freeze/thaw, mint-side `SetAuthority`, and withdrawal-config changes. Normal channel transactions containing those operations are rejected unless the configured administrator signs, while token-account owner/delegate changes remain available to users. A retired mint authority cannot silently mint private liabilities and redeem escrow.

3.2 Medium Risk

3.2.1 Withdrawals settle before Solana finality

Severity: Medium Risk.

Context: transaction.rs#L658-L688, transaction.rs#L725-L741, transaction.rs#L926-L966.

Description: The sender must keep a withdrawal recoverable until its Solana `ReleaseFunds` transaction is finalized. `send_and_confirm` instead polls with `CommitmentConfig::confirmed()` and passes any matching result to `handle_success`. That function removes the nonce from every active recovery cache and enqueues a `Completed` database update.

A transaction at confirmed commitment can still disappear from the finalized fork. In this case the user's source withdrawal has already burned their private channel tokens, but the escrow transfer is absent. The terminal row is outside ordinary stale row recovery. A restart can also find that the database claims a consumed nonce while the finalized bitmap does not, which makes the withdraw operator refuse to start until manual repair. The user therefore remains without the released funds or an automatic remint.

Recommendation: Require finalized commitment before calling `handle_success` for `ReleaseFunds` transactions. Keep the database row, persisted signature and nonce recovery state active until that commitment is reached. Use the configured weaker commitment only for operations that do not make settlement terminal.

Solana Foundation: Fixed in commit 8984c4e.

Cantina: Fix verified. `release_promotion` now treats a broadcast signature as provisional and promotes a withdrawal only after the journaled Solana transaction is finalized and error-free. Sender confirmation no longer writes `Completed`, and the promotion worker is supervised as a critical task. This restores the invariant that private settlement cannot precede irreversible release.

3.2.2 Resync can replay in flight bridge effects

Severity: Medium Risk.

Context: resync.rs#L334-L357, transaction_processor.rs#L134-L152.

Description: A destructive resync is intended to preserve evidence of every deposit mint and withdrawal remint that the private channel has already serviced. The resync takes the exclusive database live lock and builds one `ConsumedSet` from transactions visible at confirmed commitment. It later drops the database tables and uses that fixed set to decide whether rebuilt rows are terminal or `Pending`.

The live lock stops database workers after it is acquired, but it does not revoke a channel transaction that an earlier operator broadcast before crashing. Such a transaction can land after `build_consumed_set` returns. The table drop then removes its persisted attempt journal while `reconcile_against_consumed` leaves the corresponding source event `Pending` because the fixed set does not contain the late transaction. `gate_reopened_deposit` finds no surviving attempt and allows a replacement mint.

For example:

1. The operator records and broadcasts a deposit mint, then crashes before the channel confirms it.

2. Resync acquires the exclusive live lock and takes a consumed event snapshot that does not contain the mint.
3. The first mint lands after the snapshot.
4. Resync drops the attempt journal and rebuilds the source deposit as `Pending`.
5. A restarted operator broadcasts a second mint for the same deposit.

The same ordering applies to a remint broadcast for a failed withdrawal. A late remint can be omitted from the snapshot and the rebuilt withdrawal can proceed to an escrow release. The user then receives both the remint and the release. These races can create private channel liabilities without additional escrow or pay one withdrawal twice across the two chains.

Recommendation: Before dropping any table, reconcile every persisted broadcast attempt and wait until the destination chain has finalized it or advanced beyond its recorded last valid block height. Build the consumed event snapshot only after this quiescence point. Refuse the resync when any attempt remains live or ambiguous and retain its journal until its remote outcome is proven.

Solana Foundation: Fixed in commit 4dbe498.

Cantina: Fix verified. The resync wipe blocker now runs under the live operator lock and refuses destructive resync while unsettled transactions or attempt journals exist. Wiping is scoped, and the recovery marker prevents normal processing from racing partially completed recovery. In-flight bridge effects therefore cannot be erased and replayed.

3.2.3 Stalled reads exhaust gateway connections

Severity: Medium Risk.

Context: lib.rs#L1298, lib.rs#L1333-L1340.

Description: The gateway limits concurrent client connections so traffic cannot exhaust its file descriptors and memory. `handle_request` forwards each JSON RPC request with an unbounded `self.client.request(...).await`. When error redaction applies, it also collects the complete upstream body without a deadline. The connection task retains its global and per IP permits until this work completes.

If the read node accepts a request and then stops responding, an unauthenticated caller can retain one permit with any public read method. Traffic from multiple addresses can fill the global limit despite the per IP cap. The gateway then drops every new connection, including `sendTransaction` requests that route to a healthy write node. A partial read node failure can therefore make the complete public service unavailable until the stalled connections end.

Recommendation: Apply a configurable deadline to the complete upstream exchange, including response body forwarding or collection. Cancel the upstream request and return an availability error when the deadline expires so the connection permits are always released within a bounded time.

Solana Foundation: Fixed in commit f0e247b.

Cantina: Fix verified. Gateway forwarding now applies one upstream deadline across authentication, request forwarding, response-body reads, and client writes. Timeout paths return 504 and release the read permit, so a peer that stalls after headers cannot hold a connection slot indefinitely.

3.2.4 Malformed or ambiguous chain data is silently checkpointed

Severity: Medium Risk.

Context: source.rs#L2621.

Description: The Yellowstone decoder treats an invalid loaded address as a successfully handled transaction. It has equivalent fail-open behavior for an out-of-range top-level `program_id_index`, which is logged and skipped. The RPC decoder also skips transactions with missing signatures or malformed account keys instead of rejecting the slot.

These successful/empty decoder results cause the polling source to emit `ProcessorMessage::SlotComplete`. `TransactionProcessor::finalize_and_checkpoint` explicitly checkpoints empty slots, so the durable cursor advances past the omitted transaction. The fallback endpoint is only consulted when `decode_slot` returns an error; an empty successful result therefore prevents fallback recovery.

This requires malformed, truncated, or inconsistent data from Yellowstone/RPC rather than a validator-accepted malformed Solana transaction. Once it occurs, a deposit can remain in escrow without being credited, or a channel withdrawal can burn tokens without being queued for its Solana payout. Normal restart and replay begin after the advanced checkpoint, so repair requires a manual rewind or resync.

Proof of Concept: The repository already contains a minimal proof in `test_missing_signature_skips_transaction` at `indexer/src/indexer/datasource/rpc_polling/decoder.rs:617`:

1. It constructs a successful transaction whose instruction references the configured program.
2. It removes every transaction signature.
3. `parse_for_test` returns an empty instruction list rather than an error.
4. Passing the same block through `decode_slot` therefore returns `Ok(vec![])`.
5. In `RpcPollingSource`, the `Ok` arm is accepted without consulting the fallback, and `SlotComplete` is sent.
6. The transaction processor writes an empty slot and emits its checkpoint.

The same result can be reproduced on the Yellowstone path by supplying a successful `SubscribeUpdateTransactionInfo` with a loaded address whose byte length is not 32. `handle_transaction_info` logs the conversion error and returns `Ok(())`; `handle_block` subsequently sends `SlotComplete`.

Recommendation: Reject the entire slot whenever any successful transaction contains a malformed signature, account key, loaded address, program index, inner-instruction index, or lossy integer conversion. Return `DataSourceRpcError::Protocol` / `SlotRejection` and do not send `SlotComplete`.

Use checked conversions such as `u8::try_from`, validate the signature length, and consolidate validation shared by Yellowstone and RPC decoding. Add regression tests asserting that each malformed field produces an error, invokes fallback where configured, and never emits a slot checkpoint.

Solana Foundation: Fixed in commit aeedd3e.

Cantina: Fix verified. Shared transaction validation now rejects signature-count, loaded-address, instruction-index, and inner-instruction inconsistencies. Decoder errors propagate as block failures, preventing `SlotComplete` and checkpoint advancement for ambiguous input. The ingestion boundary now fails closed instead of silently accepting a partial interpretation.

3.2.5 The reconciliation halt fails open and is not atomic with transaction claims

Severity: Medium Risk.

Context: fetcher.rs#L53.

Description: The reconciliation halt is intended to freeze both bridge directions after a confirmed custody or liability shortfall. The fetcher, however, deliberately continues when `is_reconciliation_halted` returns an error. It then claims pending transactions using a separate database transaction.

`get_and_lock_pending_transactions_internal` neither locks the singleton halt row nor includes a `NOT halted` predicate. This creates both a fail-open path and a time-of-check/time-of-use window between checking the halt and changing rows from `pending` to `processing`.

The halt operation does not close this gap reliably. Persisting the durable flag and quarantining active withdrawals are independent best-effort calls. Quarantine runs only once and only affects rows present at that moment. A transient failed halt read, a failed durable halt write, or a new withdrawal inserted after quarantine can therefore reach the release pipeline while insolvency is known. No downstream component performs another halt check.

Proof of Concept: An existing test codifies the fail-open behavior: `fetcher_halt_read_error_falls_through` at `indexer/src/operator/fetcher.rs:271`.

1. Add a pending withdrawal to `MockStorage`.
2. Configure only `is_reconciliation_halted` to fail.
3. Start `run_fetcher`.
4. Despite being unable to determine whether the global halt is active, the fetcher claims the row and forwards it to the processor; the test asserts receipt of `sig_err`.

For a database-level reproduction, set the durable halt, insert a new pending withdrawal after the one-time quarantine, and make the next halt read fail transiently while the transaction-table query succeeds. The fetcher changes that row to `processing` and forwards it. The only code that checks the durable halt is the fetcher, so the release path continues.

Recommendation: Fail closed whenever the halt state cannot be read. More importantly, enforce the halt in the same database transaction that claims work:

1. Lock the singleton `reconciliation_halt` row with `SELECT ... FOR SHARE/UPDATE`.
2. Confirm that no active halt exists.
3. Select and transition pending transactions before committing the same transaction.

Alternatively, include a locked `NOT EXISTS` halt condition in the claim query. Keep retrying durable halt persistence and quarantine until both are confirmed, and add a concurrency test that pauses a fetcher between its guard and claim while another task trips the halt.

Solana Foundation: Fixed in commit 64946e5.

Cantina: Fix verified. Workers now skip claims when halt state cannot be read, and the authoritative signature/broadcast claim transaction includes the halt predicate. If the claim cannot establish a non-halted state it requeues without broadcasting. The safety stop is therefore fail-closed and atomic with ownership of bridge work.

3.2.6 Full-width token releases are truncated and create phantom liabilities

Severity: Medium Risk.

Context: transaction_processor.rs#L628.

Description: Bridge transaction amounts intentionally support the complete SPL `u64` range. The main `transactions.amount` column is therefore `NUMERIC(20,0)` and is represented by `TokenAmount(u64)`.

Observed `ReleaseFunds` events use a narrower representation: `DbObservedRelease.amount` is `Option<i64>` and `observed_releases.amount` is PostgreSQL `BIGINT`. When a valid release exceeds `i64::MAX`, the indexer silently records exactly `i64::MAX` instead of the amount transferred by the escrow program.

The reconciliation query prefers the observed amount over the withdrawal row's full-width amount. Custody consequently decreases by the actual `u64` amount while ledger withdrawals increase by only `i64::MAX`. This fabricates liabilities and, after the confirmation ticks, trips the global reconciliation halt.

Proof of Concept: Use a valid raw token amount one greater than the signed limit:

```
A = 9,223,372,036,854,775,808  // i64::MAX + 1
```

1. Deposit `A` raw units. The transaction row stores the complete value in `NUMERIC(20,0)`.
2. Withdraw the complete `A` units. `ReleaseFunds` accepts `u64`, so escrow custody becomes zero.

3. The conversion above writes `9,223,372,036,854,775,807` to `observed_releases.amount`.
4. The reconciliation SQL calculates:

```
total_deposits    = A
total_withdrawals = i64::MAX
ledger liability = A - i64::MAX = 1
custody           = 0
tolerance         = 0
```

5. The liability arm observes a non-zero custody shortfall and eventually trips the global halt, even though the payout and resulting zero custody are correct.

Recommendation: Migrate `observed_releases.amount` to `NUMERIC(20,0)` and represent it as `Option<TokenAmount>` or another checked `u64` database type. Remove saturation entirely: an out-of-range database conversion must reject the slot rather than mutate the economic value.

Add boundary tests for `i64::MAX`, `i64::MAX + 1`, and `u64::MAX`, including an end-to-end reconciliation assertion that a fully paid withdrawal leaves zero liability.

Solana Foundation: Fixed in commit 7b6d244.

Cantina: Fix verified. Token quantities are retained as full-width `u64`/numeric values through decoding, persistence, reconciliation, and release construction. Narrow signed conversions were removed or made checked, and reconciliation arithmetic avoids wraparound. Amounts above the former signed boundary no longer create a mismatch between released assets and recorded liabilities.

3.2.7 Batched AllowMint instructions can use another mint's event metadata

Severity: Medium Risk.

Context: escrow.rs#L392.

Description: `parse_escrow_instruction` receives an `InstructionLocation`, but the `ALLOW_MINT` branch discards it. `parse_allow_mint` scans every inner-instruction set in the transaction and returns the first byte sequence that resembles an `AllowMintEvent`.

The parser also does not verify the inner instruction's program ID and ignores the event's embedded `instance_seed` and `mint`; it extracts only the final decimals byte. The decoded decimals are then paired with the mint account from the currently parsed instruction.

As a result, a valid transaction containing two `AllowMint` instructions can assign the first mint's decimals to the second mint. The incorrect value is persisted in `mints`, preferred over source-chain RPC metadata, and used for just-in-time initialization on the private channel. Once the mint account is initialized, its decimals cannot be corrected in place. Balance presentation and checked token instructions then disagree with the Solana mint.

Proof of Concept: Construct one valid administrator-signed Solana transaction with:

```
top-level instruction 0: AllowMint(mint A, decimals 6)
inner set index 0:      AllowMintEvent(instance, mint A, 6)

top-level instruction 1: AllowMint(mint B, decimals 9)
inner set index 1:      AllowMintEvent(instance, mint B, 9)
```

When instruction 1 is parsed, its `InstructionLocation::top_level(1)` is not passed into `parse_allow_mint`. The loop starts at inner set 0, recognizes mint A's event, and immediately returns `decimals = 6`. `convert_to_db_models` combines those decimals with instruction 1's `accounts.mint`, producing:

```
DbMint { mint_address: B, decimals: 6, ... }
```

On mint B's first deposit, `MintCache::get_mint_metadata` finds this DB row and does not query Solana. The JIT path consequently initializes mint B on the private channel with 6 decimals instead of 9.

Recommendation: Pass `InstructionLocation` into `parse_allow_mint` and scope the search to the instruction's own top-level inner set or CPI subtree, following the approach already implemented for `parse_deposit`.

Solana Foundation: Fixed in commit 5e55da0.

Cantina: Fix verified. Allow-mint event parsing is now scoped to the same instruction execution and requires the event mint to equal the instruction mint. A neighboring instruction's event can no longer supply metadata for the current mint.

3.2.8 Unthrottled wallet challenges enable persistent database-write denial of service

Severity: Medium Risk.

Context: challenge.rs#L28.

Description: The per-IP request throttle and reduced body limit are applied only to `/auth/register` and `/auth/login`. `/auth/challenge-wallet` is added to the outer router and receives neither control.

Any valid user JWT can call the challenge endpoint repeatedly. Each request performs a username lookup and unconditionally inserts a new UUID-backed row. The schema has no one-active-challenge or per-user uniqueness constraint, so one user can create an arbitrary number of live challenge rows for the same pubkey.

Challenges expire after ten minutes, but cleanup sleeps for an hour before executing one unbounded `DELETE`. A sustained request stream can therefore saturate the auth connection pool, grow the table and indexes, generate substantial WAL and vacuum work, and cause an expensive bulk cleanup. In the supplied Compose topology, auth and the channel write node use the same PostgreSQL primary and database, extending the availability impact to ledger writes.

Proof of Concept: In an isolated test deployment, register and log in once, then use the resulting 24-hour JWT to issue a bounded concurrent request set:

```
export TEST_AUTH_URL=http://127.0.0.1:3001
export TEST_JWT='replace-with-test-user-jwt'

seq 1 10000 | xargs -P 64 -I{} curl -sS -o /dev/null \
  -H "Authorization: Bearer ${TEST_JWT}" \
  -H 'Content-Type: application/json' \
  -d '{"pubkey":"11111111111111111111111111111111"}' \
  "${TEST_AUTH_URL}/auth/challenge-wallet"
```

The requests are not constrained by `AUTH_RATE_LIMIT_PER_SECOND`. Verify the persistent amplification with:

```
SELECT count(*)
FROM private_channel_auth.challenges
WHERE user_id = '<test-user-uuid>';
```

The count grows approximately once per successful request, including repeated challenges for the same user and pubkey. Depending on when the hourly cleanup last ran, expired rows can remain for almost seventy minutes.

Recommendation: Apply request throttling to every authentication mutation route, including per-IP and per-user limits for challenge and verification requests. Store the challenged pubkey and enforce one active challenge per `(user_id, pubkey)`, replacing the previous nonce with an upsert instead of inserting unlimited rows.

Run cleanup more frequently in bounded batches, monitor challenge-row and WAL growth, and isolate or resource-limit the auth database role/pool so it cannot exhaust resources required by the channel ledger.

Solana Foundation: Fixed in commit acdbb4b.

Cantina: Fix verified. Challenge creation is now behind per-IP throttling and a per-user limiter that runs before the database write. Challenge persistence uses a unique user upsert rather than unbounded inserts. Attack traffic is bounded both before CPU/database work and in stored cardinality.

3.2.9 Tiny withdrawals drain the release payer

Severity: Medium Risk.

Context: execution.rs#L228-L232, transaction_processor.rs#L635-L654, processor.rs#L506-L527, processor.rs#L557-L567, processor.rs#L944-L976, transaction.rs#L56-L69, transaction.rs#L233-L243, instruction_util.rs#L172-L204, withdraw_funds.rs#L30-L66.

Description: A withdrawal should not cost the bridge materially more to settle on Solana than the value it moves. However, `process_withdraw_funds` rejects only an amount of zero. It accepts an amount of one raw token unit and the private channel charges no transaction fee. Several `WithdrawFunds` instructions can also reuse the same accounts in one source transaction.

The indexer turns every successful instruction into a distinct withdrawal row by its instruction index. The operator then builds one `ReleaseFunds` transaction for every row. Each builder contains a single release instruction and uses the admin and operator as signers, with the admin selected as the Solana fee payer. Release submissions await confirmation, so these rows are also processed serially.

An ordinary holder of one token for an allowed six decimal mint can split it into 1,000,000 withdrawals of one raw unit. The attacker eventually receives the same aggregate amount on Solana while the bridge must fund and confirm up to 1,000,000 separate two signer Solana transactions. The resulting rows occupy the withdrawal pipeline ahead of later users. This can deplete the release payer and leave all legitimate withdrawals unavailable until the payer is replenished and the attacker generated backlog is settled.

Recommendation: Enforce a configured minimum withdrawal amount for each mint before the burn. Size the minimum so one user's token balance cannot create Solana release costs disproportionate to its value. Also charge the user an explicit withdrawal fee or reserve that funds the destination transaction so every accepted row carries its own settlement budget.

Solana Foundation: Fixed in commit ddf1129.

Cantina: Fix verified. Per-mint minimum-withdrawal and fee parameters are stored in the on-chain `AllowedMint` configuration. Release validation enforces the minimum before asset movement, and the fee is transferred before the burn. The operator can reject economically uneconomic withdrawals while preserving explicit zero-fee configuration.

3.2.10 Gasless transfers create permanent accounts

Severity: Medium Risk.

Context: execution.rs#L797-L890, transactions.rs#L39-L49.

Description: The gasless executor permits a fresh fee payer to sponsor a transaction while preventing synthesized lamports from becoming arbitrary durable state. `is_allowed_program_instruction()` admits every top level System `Transfer`. When the payer is absent, `GaslessCallback` presents it to the SVM as a System account with ten lamports. `enforce_lamport_conservation()` then counts

each previously unknown writable account with a nonzero balance as `created`. It accepts the transaction when the fabricated payer's shortfall is no greater than this count and no older account gained value. The function clears the fabricated payer but leaves every created account unchanged.

`sendTransaction` is a public gateway method, so a caller can submit ten one lamport System transfers from a fresh payer to ten fresh destinations in one transaction. The SVM executes the transfers against the synthesized balance. The conservation check observes a shortfall of ten and ten created accounts, accepts the result and leaves the destinations live. The settler collects every successful writable account with a nonzero balance and `write_batch()` upserts it into PostgreSQL.

The channel charges no rent and has no cleanup path for these account rows. Repeating the transaction with new keys therefore adds accounts, transactions and address index records without requiring the caller to fund any state. The gateway's per IP rate limit slows this growth but does not bound accumulated storage or coordinated callers. Sustained submissions can exhaust database storage or degrade every service that depends on the store.

Recommendation: Reject a top level System transfer when its source account exists only through gasless fee payer synthesis. Continue allowing a fresh fee payer to sponsor a System transfer from a real source account. This preserves sponsored transfers and the one lamport allowance for accounts created by the allowlisted stateful programs while preventing fabricated balances from creating arbitrary System accounts.

Solana Foundation: Fixed in commit cc7c53f.

Cantina: Fix verified. Ingress validation rejects System Program instructions in send and simulate paths, preventing user-directed account allocation. Settlement also enforces conservation, burns newly created empty System accounts, and caps account creation, so bypasses cannot leave operator-funded permanent accounts.

3.2.11 Hook errors permanently block bitmap rotation

Severity: Medium Risk.

Context: proof.rs#L172-L255, proof.rs#L381-L399, transaction.rs#L477-L484, transaction.rs#L893-L906, transaction_util.rs#L221-L239, db.rs#L2937-L2971, release_funds.rs#L153-L164.

Description: The withdrawal sender keeps each release nonce in `in_flight_withdrawals` until the release either succeeds or enters the failure and compensation path. Bitmap rotation waits for this set to clear because rotating would erase the evidence needed to settle those releases. The durable unreleased nonce bounds enforce the same ordering after a restart.

`ReleaseFunds` forwards the mint's remaining accounts into the Token 2022 transfer and propagates any transfer hook error. A hook program can return `ProgramError::Custom(14)`, which Solana reports as a custom failure of the outer escrow instruction. `parse_program_error` maps every custom error 14 to the escrow program's `UnexpectedGeneration` variant without considering which operation was submitted.

`handle_confirmation_result` then handles every `UnexpectedGeneration` as a duplicate `RotateBitmap`. It only calls `clear_rotation_retry_state`, which does nothing when `ctx.kind` is `ReleaseFunds`. The handler does not remove the nonce from `in_flight_withdrawals`, update the withdrawal row or invoke `handle_permanent_failure`. Although the failed transaction rolled back both nonce consumption and the token transfer, the sender therefore retains the release as if a rotation had already settled.

An authority for an allowed transfer hook mint can point the mint at a valid hook that returns custom error 14. Any withdrawal for that mint poisons its current generation nonce. Recovery retries the failed row and eventually moves it to `ManualReview`, but that status remains part of the persistent unreleased nonce bounds. Once a withdrawal from the next generation appears, `originate_rotation_if_needed` sees both the lower owed nonce and the higher waiting nonce and refuses to arm the shared rotation. A rotation that was already armed is also withheld by the stale in memory set. The malicious mint can therefore stop all later withdrawals across every mint at the next bitmap boundary. Restarting the operator does not clear the durable blocker.

Recommendation: Treat `UnexpectedGeneration` as a settled duplicate only when `ctx.kind` is `TransactionKind::RotateBitmap`. Route the same numeric error on `ReleaseFunds` through the existing permanent failure and compensation path so the nonce leaves `in_flight_withdrawals` and the row receives a durable terminal transition. Also remap errors from downstream token and hook calls inside the escrow program to a distinct error range so a CPI failure cannot impersonate an escrow state result.

Solana Foundation: Fixed in commit c42bc08.

Cantina: Fix verified. Failure origin is now derived from the failing program's logs, so a transfer-hook custom error cannot be decoded as an escrow rotation error merely because numeric codes collide. Permanent hook refusal clears in-flight state and follows remint recovery instead of poisoning bitmap rotation.

3.2.12 Hook resolution stalls every withdrawal

Severity: Medium Risk.

Context: processor.rs#L829-L904, processor.rs#L907-L1024, mint_util.rs#L252-L327, rpc_util.rs#L89-L132, rpc_util.rs#L333-L369.

Description: The withdrawal operator should reject an unsupported transfer hook account list without delaying other mints. However, `attach_hook_extras` applies the 15 account transport cap only after `resolve_hook_extras` returns. The resolver first reads the validation account and then invokes the SPL account resolver with an RPC callback. That resolver fetches its five base accounts followed by every declared extra in sequence. Each fetch reaches the retrying `getAccountInfo` wrapper with no request budget enforced by the caller.

The hook authority of an allowlisted Token-2022 mint can change this list after the mint was reviewed. It can fill a valid oversized list with repeated references to the validation account itself. This makes every list entry trigger another download of the full list. Only after all of these reads finish does the operator learn that the resolved result exceeds its cap and park the row.

`process_release_funds` awaits this work inside its single receive loop, so it cannot process buffered withdrawals of unrelated mints at the same time. If a read fails, the error is treated as transient. The processor requeues the head row, drains the buffered rows and exits for restart before reaching the cap. The same oversized row can therefore repeat the work. An authority that can initiate a withdrawal of its mint can use a large list or repeated rows to keep the shared withdrawal processor occupied.

The documented trust decision for a transfer hook permits its authority to stop transfers of that mint. It does not grant that authority control over other supported mints. The runbook also describes an oversized list as a condition that parks every withdrawal of the affected mint, while the current ordering expands the failure to every withdrawal handled by the operator.

Recommendation: Retain the initial validation account data and parse its declared list length before resolving any entry. If the declared entries plus the hook program and validation account exceed `MAX_HOOK_EXTRAS_LEGACY_TX`, return the existing `hook_unresolvable` bail immediately. Pass the retained data into the resolver with a strict fetch budget so the known transport limit also bounds all account reads before the row is parked.

Solana Foundation: Fixed in commit 2869f3e.

Cantina: Fix verified. Transfer-hook accounts are pre-resolved and validated before the worker loop, with limits on account count and retained bytes. Deterministically invalid hook metadata parks only the affected work, while transient RPC failures remain retryable. One hostile mint can no longer indefinitely occupy the global withdrawal path.

3.2.13 Memo upgrade blocks historical release replay

Severity: Medium Risk.

Context: backfill.rs#L214-L262, escrow.rs#L24-L30, escrow.rs#L575-L605, resync.rs#L346-L383, resync.rs#L429-L439.

Description: The escrow indexer must replay every successful `ReleaseFunds` instruction for the configured instance when it rebuilds release and accounting state. Before commit `db17f68`, the bitmap era escrow program accepted 13 fixed accounts for a mint without a transfer hook. The memo upgrade added `memo_program` at index 13 but did not change the instruction discriminator, data or any account at indices 0 through 12.

The indexer changed `RELEASE_FUNDS_ACCOUNTS` from 13 to 14 with the program upgrade. `parse_release_funds` now rejects every instruction whose account vector has fewer than 14 entries, even though `ReleaseFundsAccounts` only resolves indices 0 through 12 and the indexer does not consume the memo program. A successful 13 account release recorded before the upgrade therefore returns `InsufficientAccounts` under the current binary.

`decode_slot` treats this error from a supported instruction as a rejection of the complete slot. A startup backfill cannot advance past that slot. A destructive escrow resync first drops and recreates the shared database tables, then starts the same backfill from the configured genesis slot. If the history contains a release from before the memo upgrade, the rebuild aborts after the data has been destroyed. The escrow indexer and dependent operator cannot reconstruct service from chain history until compatible parser code is deployed or the original database is restored.

Recommendation: Accept 13 accounts as the minimum historical `ReleaseFunds` layout and continue resolving the shared fields at indices 0 through 12. Treat any accounts after that prefix as version specific trailing accounts that the indexer does not need. Failed transactions from the upgraded program are already excluded before instruction parsing, so this remains compatible with the current program requirement for `memo_program` while allowing successful releases from the preceding version to replay.

Solana Foundation: Fixed in commit 22952df.

Cantina: Fix verified. The indexer's `parse_release_funds` now accepts the 13-account prefix shared by every bitmap-era release, reads only those stable accounts and ignores later memo/hook additions. Successful pre-memo releases therefore replay during backfill, while 12-account input is rejected and failed transactions never enter this parser.

3.2.14 Mint reallow leaves stale withdrawal data

Severity: Medium Risk.

Context: processor.rs#L87-L106, processor.rs#L496-L516, mint_util.rs#L69-L84, mint_util.rs#L134-L176.

Description: `AllowMint` is the documented recovery path when a closeable mint is recreated at the same address with a different profile. The instruction overwrites the existing `AllowedMint` account with the current token program and the indexer correspondingly updates the mint's `token_program` database field.

The running withdrawal operator does not observe that update. `MintCache::get_mint_metadata()` stores the token program under the mint address and serves that entry for the lifetime of the process. `ReleaseFundsState::get_instance_ata()` separately stores the escrow ATA under the mint address alone, even though the ATA derivation also includes the token program. `build_release_funds()` uses both cached values instead of the fresh `AllowedMint` profile that the processor reads immediately beforehand.

Consequently, the operator can first handle the mint and warm both caches. The mint can later close at zero supply and be recreated under the other supported token program. After the administrator allows it again, new deposits can succeed against the updated profile and create backed private balances. Each subsequent withdrawal is nevertheless built with the previous token program and previous escrow ATA. The escrow program rejects the release because the current mint is not owned by the supplied token program, so the user receives no destination tokens. The sender eventually compensates the burned private balance, but every retry is built from the same stale cache and fails again. Withdrawals for that mint remain unavailable until the Withdraw operator is restarted.

Recommendation: Build withdrawals from the token program stored in the freshly read `AllowedMint` account and derive the recipient and instance ATAs from that value. Remove the permanent metadata cache from this path or version and invalidate it whenever a new `AllowMint` profile is indexed. If ATA caching remains, key entries by both mint and token program rather than mint alone. Add a regression that warms the legacy profile, indexes a Token-2022 profile and verifies that the same running operator builds the next release with the new token program and ATAs.

Solana Foundation: Fixed in commit d4ae52b.

Cantina: Fix verified. Withdrawal processing now rereads the current `AllowedMint` token-program identity for each row and freshly derives the relevant PDAs and associated token accounts. Stale cached mint data is no longer reused after block/reallow changes.

3.2.15 Same slot blocks suppress valid deposits

Severity: Medium Risk.

Context: transaction_processor.rs#L247-L255, processor.rs#L1156-L1167, processor.rs#L1221-L1239, db.rs#L784-L791, db.rs#L2714-L2721, db.rs#L2736-L2747.

Description: The deposit operator is intended to mint a private balance when an authorized Solana deposit transfers tokens into escrow. The indexer records each `AllowMint` or `BlockMint` transition with only its containing slot. The database keys this history by `(mint_address, effective_slot)` and the authorization query selects the latest status whose slot is no later than the deposit slot. Mint status rows do not retain a transaction or instruction position within the block, even though deposit rows retain their source instruction position.

This representation can apply a later block retroactively. Consider a mint that was already allowed before slot S. A user successfully executes `Deposit` early in slot S, which means the on chain mint gate accepts the operation and transfers the user's tokens into escrow. The administrator then executes one routine `BlockMint` later in slot S. The indexer records the block with `effective_slot = S`, so `assert_mint_allowed_at_slot()` returns blocked for the earlier deposit from that same slot.

`process_deposit_funds()` consequently moves the deposit to `ManualReview` and sends no `MintTo` request to the private channel. The user's tokens remain in escrow without the matching private balance. Allowing the mint in a later slot does not repair the historical lookup because the later allow is after the deposit slot. Rearming the row therefore repeats the refusal until operators perform a bespoke historical repair or issue an independently verified manual credit. A safety block during active deposit traffic can affect every deposit for that mint that executed earlier in the block's slot.

Recommendation: Persist a total source position within the block for mint status changes and deposit instructions. Resolve the latest mint status whose full position is no later than the deposit's position. Preserve this ordering through both data sources and replay. Migrate existing history conservatively so deposits and status changes from the same legacy slot require explicit review rather than silently applying the final slot state retroactively.

Solana Foundation: Fixed in commit dbb91e6.

Cantina: Fix verified. Mint-status history now records and queries transaction/block position within a slot rather than collapsing all changes to one row per slot. Deposit authorization uses the exact same-slot ordering, so a later block event cannot retroactively invalidate an earlier valid deposit.

3.2.16 V1 upgrade rejects prior transaction rows

Severity: Medium Risk.

Context: types.rs#L394-L431.

Description: PostgreSQL is the source of truth for transaction history, so a node upgrade must preserve rows written by the preceding supported release. Before commit `a6375e7`, the settler stored each `StoredTransaction` directly with `bincode`. The v1 transaction upgrade replaces that representation with an `SPCT` header followed by a new metadata layout and exact signed transaction bytes.

`StoredTransaction::from_bytes` now rejects every row that lacks the new header as `UnknownFormat`. It has no branch for the preceding `bincode` layout and startup does not migrate existing rows. New settlement writes only use the new representation, leaving every retained row from before the upgrade unchanged in PostgreSQL.

An operator triggers the failure by rolling the binary forward against an existing database. Both `getTransaction` and `getSignatureStatuses` then return server errors for all earlier signatures. `getBlock` also errors when a requested block includes one of those signatures and `getSignaturesForAddress` fails any page that includes an earlier transaction. The latter endpoint is required to enumerate serviced mints before an indexer resync, so the format change also prevents the supported bridge recovery procedure from rebuilding a database that has prior channel activity. If the address index watermark trails the tip during the upgrade, the write node encounters the same decoder error in its mandatory index repair and cannot start.

Recommendation: Add an exact decoder for the preceding headerless `StoredTransaction` layout and translate its metadata into the current in memory type. Migrate every legacy PostgreSQL row to the versioned format while holding the writer lease and retain the compatibility decoder until the migration has completed on the primary and its replicas.

Solana Foundation: Acknowledged. Private channels have no production deployment that predates #310, so every supported deployment starts with the SPCT row format and no legacy headerless rows exist to decode.

3.2.17 Token account closures can squat future mints

Severity: Medium Risk.

Context: transactions.rs#L46-L55, admin.rs#L77-L115, admin.rs#L345-L361, auth.rs#L63-L94, mint.rs#L134-L188, transaction.rs#L759-L852.

Description: Private channel mints use the same public keys as their mints on the source chain, but they are not created until the first deposit needs to mint private tokens. Before that initialization, the public `sendTransaction` method admits every SPL Token and Associated Token Account instruction. A user can create an empty ATA for any existing private mint, then close that ATA with a future mint public key as the close destination. SPL Token does not require the destination to sign. Closing the ATA therefore moves its single lamport existence balance into a new empty account owned by the System Program at the chosen future mint address.

The first deposit for the squatted mint has already transferred the user's source tokens into escrow when the operator submits its private channel `MintTo`. That transaction fails with `IncorrectProgramId`, which the sender classifies as `MintNotInitialized`. The JIT helper then reads the existing account successfully, fails to decode its empty data as an SPL Mint and moves the deposit to `ManualReview` without attempting initialization. Even a direct retry cannot repair the state. `AdminVm::check_initialize_mint_target()` rejects every existing account not owned by SPL Token with `ExternalAccountDataModified`.

An anonymous caller can preempt any known source mint before its first private channel deposit, including before the mint is allowlisted. The same ATA creation and closure lifecycle can be repeated for additional targets without holding any tokens. Every later deposit for a squatted mint remains unserviceable, while each depositor's source tokens remain locked in escrow until an operator coordinates a manual refund or performs unsupported state repair. Other mints continue operating, so the impact is a targeted persistent denial of service and custody lock rather than a halt across the channel.

This is distinct from `medium_gasless-transfers-create-permanent-accounts.md`. That issue converts a synthetic fee payer directly into arbitrary System accounts and impacts database capacity. Its recommendation rejects System transfers sourced from synthetic payers. This attack uses the intended gasless ATA existence floor and a standard `CloseAccount` from a token account that already exists, so that recommendation does not protect the deterministic mint namespace. Its violated invariant and impact are also different: an authorized source mint must remain initializable before accepting backed deposits.

Recommendation: Reserve or initialize every private mint before its allowlist on the source chain can accept deposits. Make readiness conditional on verifying the private account's owner, decimals and mint authority. To make recovery safe for deployed channels, define one narrow representation for a reclaimable mint reservation, such as an empty account owned by the System Program at a mint address authorized by an admin. Handle that representation consistently in both the JIT precheck and AdminVM. Do not broadly overwrite accounts owned by another program or accounts that contain data. Alternatively, move private mints into a namespace that public transactions cannot occupy and maintain an explicit mapping from source mints to private mints.

Solana Foundation: Fixed in commit cc7c53f.

Cantina: Fix verified. In addition to rejecting System Program instructions at ingress, settlement now detects and burns newly created empty System accounts and limits account creation. Closing a token account cannot leave a lamport-funded System account that reserves the address for a later mint.

3.2.18 Async address index reopens serviced deposits

Severity: Medium Risk.

Context: get_signatures_for_address.rs#L92-L160, write_batch.rs#L186-L219, write_batch.rs#L396-L401, get_signatures_for_address_impl.rs#L33-L83, address_index_writer.rs#L26-L34, address_index_writer.rs#L80-L125, address_index_writer.rs#L173-L210, settle.rs#L1400-L1462, resync.rs#L100-L135, resync.rs#L332-L407, transaction_processor.rs#L126-L154, processor.rs#L1132-L1205, mint.rs#L429-L484.

Description: Resync builds a consumed mint set before it deletes the indexer database. This set is meant to ensure that every deposit already minted on the private channel is rebuilt in its terminal state. The core can return a successful but incomplete authority history, so this check can omit a finalized mint.

`write_batch` commits each transaction, block and advertised tip in PostgreSQL before it returns the corresponding address rows to the settler. The settler publishes the committed accounts and blockhash, then hands those rows to a separate address index writer. That writer uses its own database transaction and can trail the source transaction during ordinary scheduling, database slowdown or its retry sequence. The queue admits up to 512,000 pending rows and the configured retry delays total 9.25 seconds before the final attempt.

`getSignaturesForAddress` queries only the `address_signatures` table. It does not read `address_signatures_flushed_slot` or compare that watermark with the advertised tip. It therefore returns a normal short or empty page while recent transactions remain pending in the writer. At the same time, `getSignatureStatuses` reads the committed transaction table and reports the same mint as finalized, which lets the operator mark the source deposit complete and drain the ordinary work queue.

During a later authorized resync, `enumerate_consumed_mints` treats that short history as complete. Resync then drops the database and injects the incomplete set into the rebuild. The omitted source event is recreated as `Pending`, while the drop also removes its prior attempt journal. The operator can consequently submit another `MintTo` for the same deposit.

This requires an authorized operator to start destructive resync while the address index trails the committed tip. An ordinary user cannot request that operation directly, but a completed deposit that falls in the lag window can receive a second private balance for the same escrowed tokens. The duplicate increases private liabilities without adding custody and can leave the pooled escrow unable to honor all withdrawals.

Recommendation: Expose the durable `address_signatures_flushed_slot` through the channel RPC. At resync, capture a fixed channel tip, require the watermark to reach that tip and constrain consumed mint enumeration to the captured tip before dropping any tables. Treat a trailing or unavailable watermark as a preflight failure.

Solana Foundation: Fixed in commit 7b3d7ba.

Cantina: Fix verified. Address-index updates and their watermark advance atomically with each processed block. `getAddressIndexSlot` combines the durable watermark with the indexed tip, and

resync refuses or waits while coverage is incomplete. A lagging asynchronous index can no longer be mistaken for complete history.

3.2.19 Bulk block queries overwhelm ordinary reads

Severity: Medium Risk.

Context: get_blocks.rs#L42-L58, get_blocks_with_limit.rs#L42-L58, postgres.rs#L11-L27, postgres.rs#L73-L80, get_blocks_impl.rs#L12-L37, get_blocks_with_limit_impl.rs#L10-L26, auth.rs#L64-L94, lib.rs#L233-L245, lib.rs#L1080-L1098.

Description: The gateway limits public request rates and connections so hostile traffic cannot exhaust shared service resources. However, it spends one rate limit token before considering the method or the amount of work requested. `getBlocks` and `getBlocksWithLimit` are available without authentication and receive the same token cost as a constant cost read.

Both methods accept up to 500,000 slots or returned blocks. Their PostgreSQL implementations use `fetch_all` to materialize every matching row before the RPC layer allocates and serializes the complete result. One small public request can therefore hold a database connection while it fetches 500,000 rows and produces a response of several megabytes.

The default gateway configuration permits 64 simultaneous connections from one IP with a burst of 100 requests and a sustained rate of 50 requests per second. The core PostgreSQL pool has 32 connections by default. A single unauthenticated caller can consequently place one full wave of bulk scans on every pool connection and queue another wave ahead of ordinary reads. Continued requests can maintain the pressure.

In the audit benchmark, one supported `getBlocks` call returned 500,000 rows and a 3,388,925 byte response. Eight concurrent calls against a four connection pool reproduced the production ratio between the per IP connection allowance and pool size. An unrelated `getSlot` call increased from 5.8 milliseconds to 5.0 seconds while the bulk calls returned 27,111,400 bytes in total. On the default limits, one IP can request 32 million rows in its initial concurrent burst. This lets an anonymous caller cause meaningful latency and availability loss for ordinary RPC clients while consuming PostgreSQL capacity.

Recommendation: Apply a separate work budget to block enumeration. Charge the public quota in proportion to the requested range or limit and cap concurrent enumeration calls below the PostgreSQL pool size so ordinary reads always retain capacity. Read and serialize results in bounded pages instead of materializing as many as 500,000 rows at once.

Solana Foundation: Fixed in commit acd2b31.

Cantina: Fix verified. `get_blocks_impl` and `get_blocks_with_limit_impl` acquire a pool sized permit before large scans. `Gateway::handle_request` separately limits public listings and rejects excess calls immediately. The per client guard remains through the response body. With the default pool and matching gateway configuration, bulk listings leave capacity for ordinary reads.

3.2.20 Expired mint transactions can execute twice

Severity: Medium Risk.

Context: dedup.rs#L124-L133, execution.rs#L927-L960, execution.rs#L1098-L1113, processor.rs#L1040-L1053, processor.rs#L1129-L1205, recovery.rs#L551-L585, remint.rs#L632-L736.

Description: The channel advertises a `last_valid_block_height` for each recent blockhash. The deposit operator relies on that deadline when recovering a mint whose signature is still absent. Once the height has passed, `classify_signatures()` returns `Dead`, `check_deposit()` reports `NotLanded` and the row can be rebuilt with a fresh blockhash. This is safe only if a transaction cannot execute after its hash leaves the live window.

The executor does not enforce that property atomically. Its final `retain_live()` call copies `live_blockhashes` into a local `HashSet` and immediately releases the read lock. The dedup task can then add a settled blockhash and evict the transaction's hash while the executor filters the batch, partitions transactions, refreshes the clock and runs the SVM. The source acknowledges this race and relies

on the interval before dispatch being short. The SVM receives successful prechecks and a default processing blockhash, so it does not repeat the validity check.

This permits the following sequence when the executor is delayed after its final snapshot or while the accepted batch is still producing a visible result:

1. The operator broadcasts a deposit `MintTo`, records its signature and the transaction enters an execution batch while its blockhash is live.
2. The executor snapshots the live set. A later settled block evicts the hash before the mint has a visible status.
3. Recovery observes a null signature after `last_valid_block_height`, classifies it as dead and reopens the deposit. The reopened deposit gate reaches the same `NotLanded` decision and sends a replacement signed over a fresh blockhash.
4. The original transaction executes from the stale snapshot. The replacement has a different message hash and can execute afterward. The idempotency memo is only an observation marker and does not stop SPL Token from applying both `MintTo` instructions.

The recipient is credited twice even though escrow received one deposit. Reconciliation can later detect that channel supply exceeds backing and halt the bridge, but it cannot reverse tokens that the recipient has already transferred or exchanged.

Recommendation: Keep the final live blockhash read guard until both virtual machines have finished and their successful account updates have been applied to BOB. This makes eviction, the public validity answer and execution admission share one order. A hash must not become invalid while an accepted transaction can still produce a new state transition or an unpublished successful result.

Solana Foundation: Fixed in commit a7f5870.

Cantina: Fix verified. A `BlockhashProgress` barrier now orders blockhash retirement after every transaction admitted under that hash has settled and committed. Status updates and block-height/floor advancement are atomic, eliminating the window where a retry can be admitted while the first execution is still unrecorded.

3.2.21 Gap repair can regress mint metadata

Severity: Medium Risk.

Context: source.rs#L640-L650, source.rs#L678-L791, transaction_processor.rs#L230-L335, transaction_processor.rs#L575-L594, processor.rs#L496-L516, mint.rs#L190-L224, mint_util.rs#L134-L176.

Description: The Yellowstone indexer repairs finalized slots that were missed during a connection outage while the resumed live stream continues. The first finalized Slot update on a new connection fixes the repair target and spawns the RPC fill. Later live Blocks are forwarded through the same processor without waiting for that fill. The checkpoint gate prevents the durable frontier from crossing the missed range, but it does not delay the database writes for those later blocks.

`TransactionProcessor::finalize_and_checkpoint()` processes each completed slot as it arrives. An `AllowMint` becomes a `DbMint` containing `decimals` and `token_program` without its source slot. A conflict on the mint address replaces those fields with the incoming values. Therefore, an older `AllowMint` delivered by the concurrent repair can overwrite a later live profile. Mint status history avoids the same problem by storing `effective_slot` and selecting the latest transition, but the profile fields have no equivalent ordering information.

The failure is reachable when an administrator follows the documented recovery flow for a closeable mint:

1. The indexer disconnects after checkpoint 100 and misses an existing `AllowMint` at slot 103. On reconnect, its first Slot update arms repair only through slot 103.
2. The zero supply source mint is closed, recreated with different decimals or under the other supported token program and allowed again at live slot 106. The live event writes the current profile first.

3. The slower repair later reaches slot 103 and replaces the current profile with the obsolete values. A subsequent live slot advances the checkpoint past slot 106, so an indexer restart does not replay the current event.
4. An ordinary user deposits against the current on chain profile. On the first deposit for an asset without an initialized private mint, the JIT path reads the obsolete database decimals and permanently initializes that mint with the wrong denomination. If the source token program changed, the withdrawal operator instead derives the release accounts from the obsolete program and cannot release that asset until the indexed profile is repaired.

Raw token units remain conserved and the trigger requires a Yellowstone reconnect to overlap the administrator's profile change. However, the resulting database state is persistent and affects every user of that mint. A decimal mismatch can make wallets and integrations interpret balances by a different factor on each chain. A token program mismatch makes withdrawals for the mint unavailable even after an operator restart.

Recommendation: Make mint profile persistence monotonic in source order. Carry the source slot and available instruction position into `DbMint`, store that position with the profile and update `decimals` and `token_program` only when the incoming profile is newer than the stored one. Do not let the checkpoint gate serve as the ordering guarantee for row writes.

Rebuild existing mint rows from their latest indexed `AllowMint` transition before resuming deposits and invalidate any operator metadata cached from an older profile.

Solana Foundation: Fixed in commit d4ae52b.

Cantina: Fix verified. Mint profile persistence uses a monotonic `profile_slot` upsert with deterministic same-slot ordering, and withdrawal workers reread the fresh profile. Older gap-repair observations cannot overwrite newer token-program or mint metadata.

3.2.22 Gap repair can strand authorized deposits

Severity: Medium Risk.

Context: source.rs#L640-L650, source.rs#L678-L791, transaction_processor.rs#L230-L347, fetcher.rs#L18-L105, processor.rs#L1117-L1240, recovery.rs#L178-L267.

Description: The Yellowstone source repairs finalized slots missed during a reconnect while it continues forwarding later live blocks. The first finalized Slot update fixes the repair target, but a later live block can reach `TransactionProcessor` and write its transactions before the slower RPC repair supplies an older `AllowMint`. The checkpoint gate delays the durable frontier. It does not prevent the separately running operator from consuming rows that were written beyond that frontier.

This creates a persistent false refusal for an ordinary deposit. Suppose the indexer reconnects at checkpoint 100 and its first Slot update starts repair through slot 103, which contains the authentic `AllowMint` for an asset. While that fill is unresolved, a user deposits the now authorized asset at live slot 106. The live transaction is inserted as `Pending`, the fetcher immediately claims it and `assert_mint_allowed_at_slot()` queries an authorization history that does not yet contain slot 103. It treats `NeverAllowed` as a permanent `MintNotAllowed` error. `process_deposit_funds()` therefore moves the row to `ManualReview` and sends no private channel `MintTo`, even though the source program already transferred the user's tokens into escrow.

When repair later inserts the slot 103 authorization, the indexed history correctly reports that the deposit was allowed. Nothing revisits the terminal deposit row. The normal fetcher selects only `Pending` rows. Periodic recovery rechecks `ManualReview` rows only for withdrawals. The documented operator procedure can diagnose and rearm each affected deposit, but until that manual action the user has neither the source tokens nor the corresponding private balance. Every deposit for the newly allowed mint that is claimed during the repair window can be stranded this way. The trigger requires a Yellowstone reconnect and a repair delay overlapping a recent mint authorization. An ordinary user cannot create those infrastructure conditions on demand.

Recommendation: Do not expose or claim a live deposit until the indexer's complete repair frontier covers the deposit's slot. Alternatively, make the authorization lookup return a distinct incomplete history result whenever the durable checkpoint or repair target trails the deposit and retry that row instead of quarantining it.

After the repair frontier advances, automatically reevaluate quarantined deposits that have no private channel attempt. Requeue only those whose repaired history proves they were allowed at their source slot. Keep a true `NeverAllowed` result terminal once history completeness for that slot is established.

Solana Foundation: Fixed in commit 56d0d9a.

Cantina: Fix verified. A deposit database claim is now gated on the durable escrow checkpoint reaching the deposit's slot. Uncovered deposits remain pending in the backlog instead of being classified against incomplete allow/block history, so gap repair cannot permanently discard an authorized credit.

3.2.23 Native SOL deposits remain locked in escrow

Severity: Medium Risk.

Context: processor.rs#L1144-L1204, transaction.rs#L907-L913, transaction.rs#L1517-L1538, transaction.rs#L1568-L1571, transaction.rs#L2428-L2451, instruction_util.rs#L386-L453, allow_mint.rs#L68-L99, deposit.rs#L113-L166.

Description: `AllowMint` is intended to select source assets whose deposits the private channel can represent. It reads the mint profile and creates the escrow ATA without excluding `spl_token::native_mint::id()`. The supplied command examples use this native mint address. After an admin allows it, an ordinary holder can deposit wrapped SOL. `Deposit` transfers the requested units into the instance ATA and emits a `DepositEvent` for the amount received.

The operator preserves the source mint address when it builds the private receipt. `MintToBuilder` first creates the recipient ATA for that address and then issues `MintTo`. SPL Token marks an account as native based on the mint address, so the recipient ATA is a native token account. SPL Token rejects `MintTo` for such an account with `NativeNotSupported` even though the private mint itself was initialized successfully.

The sender treats this error as a permanent program failure and records the deposit as `Failed`. The source transfer has already finalized in a separate transaction and deposits have no automatic refund path. The deposited wrapped SOL therefore remains in escrow while the user receives no private receipt. Recovering it requires an operator to verify the failed deposit and submit a privileged `ReleaseFunds` refund.

This affects the one native asset after the trusted admin enables its documented configuration. It is distinct from `fabricated-wsol-buys-backed-tokens.md`, which creates value from synthetic private lamports and trades it for backed tokens. Fixing that conservation error does not make a native ATA accept `MintTo`. Here the incompatible source and private representations deterministically lock a legitimate deposit.

Recommendation: Reject `spl_token::native_mint::id()` in `process_allow_mint` before creating its escrow ATA. The operator should also refuse any existing native mint allowlist entry and prevent further deposits until affected users have been refunded with `ReleaseFunds`.

If native SOL support is required, map the source native mint to a separate private mint that supports ordinary `MintTo` and `Burn`. Apply the reverse mapping when building withdrawals so the private receipt never relies on native token account semantics.

Solana Foundation: Fixed in commit e76ac25.

Cantina: Fix verified. The escrow `AllowMint` processor and channel ingress both reject the canonical native SOL mint. Users therefore cannot enter the unsupported path that deposits native-mint balances without a viable private-channel lifecycle.

3.2.24 Read nodes report live blockhashes expired

Severity: Medium Risk.

Context: is_blockhash_valid_impl.rs#L16-L45, lib.rs#L1243-L1249.

Description: `isBlockhashValid` is intended to tell clients whether the write node can still execute a transaction that names a given recent blockhash. The gateway routes this call to the read service because it sends every method other than `sendTransaction` to `read_url`. The RPC handler then checks only `ReadDeps.live_blockhashes`, a process local list that is maintained by the writer's Dedup stage.

The shipped read service does not run Dedup. It starts with an empty `live_blockhashes` list and has no task that populates it from PostgreSQL or from the writer. The handler therefore returns `false` for every blockhash for the lifetime of the read process. This includes a hash that the same service has just returned from `getLatestBlockhash` and that the write node still accepts.

A client or relayer that consumes this public Solana compatible response can treat a pending transaction as expired while its hash remains live. If it signs the same nonidempotent operation over a later blockhash, the changed recent hash produces a different message identity and signature. Both variants pass the writer's live hash and message deduplication checks, so both effects can settle. This can duplicate a transfer or another client initiated value operation. Clients that validate a hash before submission can instead refuse every transaction.

This consequence requires the client or application to act on the false answer and sign a replacement before it observes the original result. The current bridge operator does not call `isBlockhashValid`, so the defect does not by itself trigger an automatic duplicate deposit mint or compensation remint.

Recommendation: Expose the writer's authoritative Dedup live window through `isBlockhashValid` and route this RPC to that writer backed endpoint while keeping ordinary reads on the read service. Return an RPC error when the authoritative validity state is unavailable rather than answering from an empty or unsynchronized process local list.

Solana Foundation: Fixed in commit 8984c4e.

Cantina: Fix verified. `isBlockhashValid` is routed to the write node and evaluated against the deduplicated live blockhash window. Catch-up or uncertain states now return a retryable error instead of a false expiry, keeping read-node lag from contradicting admission.

3.2.25 Inconsistent finality reads can duplicate mints

Severity: Medium Risk.

Context: get_signature_statuses_impl.rs#L30-L38, processor.rs#L1040-L1053, processor.rs#L1135-L1205, recovery.rs#L556-L585, recovery.rs#L907-L920, remint.rs#L61-L225, remint.rs#L497-L532, remint.rs#L632-L733.

Description: The operator journals every channel mint signature before broadcast. It should create a replacement only after it proves that all prior attempts are absent and cannot land.

`classify_endpoint()` asks `getSignatureStatuses` for the journaled signatures but discards `response.context.slot`. It then makes separate `getBlockHeight` and `getFirstAvailableBlock` calls. The channel RPC deliberately reads its context slot before looking up each transaction so a null status cannot claim to cover a later slot. Ignoring that context lets the classifier combine a stale null with newer height and retention answers into a false `Dead` verdict.

The compensation remint path can duplicate a mint on the default single replica topology:

1. A remint after a permanent withdrawal failure finalizes on the channel but the operator loses the acknowledgement and retains the journaled attempt.
2. During retry, the replica returns a null status at a context slot below the attempt's journaled `blockhash_slot`.
3. The replica catches up before the following calls. Its height is now past the attempt's `last_valid_block_height` and its ledger floor covers the attempt's slot, so `classify_signatures()` returns `Dead`.

4. `attempt_remint()` retires the prior entry then signs and broadcasts a new compensation mint without another finality check.

Both remints compensate one failed withdrawal, so the user receives more channel tokens than were burned.

The same classifier can duplicate a deposit mint in the documented horizontal deployment. Recovery can combine a lagging status replica with current height and floor replicas then change a finalized deposit from `Processing` to `Pending`. The processor performs another finality check before minting. If load balancer routing repeats the inconsistent proof while the lag persists, it creates a fresh `MintTo` transaction for the same escrow deposit. A single monotonic replica self heals this deposit path at the second check but the compensation path has no second check.

The replacement uses a new blockhash and signature, so transaction hash deduplication does not reject it. The extra tokens create unbacked channel supply. Reconciliation can later halt bridge processing but it cannot reverse tokens that the recipient already transferred.

Recommendation: Classify an absence through one writer backed internal RPC that reads the transaction status, current height, ledger floor and response context from one repeatable read snapshot. Return `Dead` only when that snapshot covers every journaled `blockhash_slot`, is past each `last_valid_block_height` and still retains each attempt window.

Solana Foundation: Fixed in commit 164b58f.

Cantina: Fix verified. `getSignatureStatusSnapshot` now reads signature status, chain height, and retention floor in one repeatable-read database snapshot. Missing height or records older than the core retention floor produce `Uncertain`, so the operator cannot combine mutually inconsistent reads into a duplicate-mint decision.

3.2.26 Replica skew leaks data after account handoffs

Severity: Medium Risk.

Context: auth.rs#L67-L113, auth.rs#L369-L452, auth.rs#L483-L572, lib.rs#L616-L717, lib.rs#L914-L947, lib.rs#L1235-L1299.

Description: The account gated RPC methods are intended to return token account state and history only for periods when a verified wallet controls that account. The gateway decides this policy before it independently forwards the client request, so the authorization and returned data can come from different ledger snapshots.

For `getAccountInfo` and `getTokenAccountBalance`, `fetch_account_for_auth()` issues a hidden `getAccountInfo` request. `check_token_account_ownership()` authorizes the caller from the owner or delegate fields in those returned bytes. The gateway then forwards the original call to `read_url` and returns its response without checking that its context or authority matches the snapshot used for authorization.

`getSignaturesForAddress` adds an ownership range but retains the same gap. `resolve_owned_slot_ranges()` reads the recorded owner changes separately and gives the apparent current owner an open ended range through `i64::MAX`. When no handoff has ever been seen it can return no range restriction at all. The fixed policy is then attached to another request. A handoff that becomes visible after policy resolution is outside the gateway's proof, yet the later backend history query can return signatures above that handoff.

The supported horizontal deployment places multiple read replicas behind a load balancer, so the requests can observe different ledger frontiers. A previous account owner can exploit this during replica lag:

1. The token account is handed to a new owner while the previous owner's wallet remains registered with their gateway user.
2. The authorization fetch and ownership range resolution observe the previous owner's view.
3. The forwarded account or history call observes a newer view after the handoff.

4. The gateway returns either the new owner's current account bytes or balance or post handoff history metadata to the previous owner.

A revoked delegate can use the current state variants, while history is owner only. The history race can also occur on one advancing backend if the handoff lands between range resolution and the history query. Replica divergence widens that window. The leaked history is limited to the signature page fields, including signatures, slots, memos, timing, confirmation state and success versus redacted failure. It does not grant `getTransaction` access or expose full transactions. The current state variants expose the full token account bytes and balance.

Recommendation: Authorize the exact ledger snapshot used to construct every client response. For `getAccountInfo`, inspect the account bytes in the forwarded response before returning them. For `getTokenAccountBalance`, have the backend return the account authority with the balance from one consistent snapshot or derive the balance from the same account bytes that passed the ownership check.

For `getSignaturesForAddress`, retain the context slot from the authorization snapshot and cap every open ended ownership range at that slot. Another option is to query the ownership chain and address history in one authoritative database snapshot. Recheck owner changes through the newest returned slot before releasing a page. Fail closed whenever the gateway cannot bind the response to the ownership proof.

Solana Foundation: Fixed in commit e8b428e.

Cantina: Fix verified. Gateway authorization captures an account snapshot and rechecks returned account bytes against it; balance replies are derived from authorized bytes. Ownership history and tip are read in a single SQL snapshot with bounded windows. Replica skew can no longer authorize one owner and return another owner's state.

3.2.27 Slow webhooks corrupt recovered transactions

Severity: Medium Risk.

Context: db_transaction_writer.rs#L33-L35, db_transaction_writer.rs#L48-L52, db_transaction_writer.rs#L60-L81, db_transaction_writer.rs#L92-L110, db_transaction_writer.rs#L127-L136, operator.rs#L128-L145, processor.rs#L182-L223, processor.rs#L624-L653, processor.rs#L921-L970, recovery.rs#L26-L33, recovery.rs#L202-L225, recovery.rs#L245-L405, recovery.rs#L907-L920, proof.rs#L172-L263, state.rs#L425-L559, types.rs#L102-L116, db.rs#L1600-L1631, db.rs#L1969-L2060, db.rs#L2439-L2456, db.rs#L2936-L2971, withdraw_funds.rs#L25-L68.

Description: Transaction outcomes should become durable independently of alert delivery. `DbTransactionWriter::start` instead processes one status update at a time. After it writes a `Failed` status, a `FailedReminted` status or a `ManualReview` status, it awaits the alert webhook before reading the next update. A configured endpoint that accepts a connection without replying can therefore hold the only writer for its full ten second timeout on every alert. The fixed status channel holds 100 messages and producers use `send_guaranteed`, so a sustained alert burst also blocks the operator once that channel fills.

Queued outcomes are not bound to the transaction incarnation that produced them. `TransactionStatusUpdate` carries the row ID but no captured `updated_at` value. `update_transaction_status_internal` accepts an update whenever that row is currently `processing` or `pending_remint`, even if recovery has already requeued it and the fetcher has locked a newer incarnation.

An ordinary holder can reach the alert path by submitting withdrawals for a mint whose escrow allowlist account was removed or whose withdrawals were blocked. The source program accepts any nonzero burn without consulting the escrow allowlist. The withdrawal processor then leaves each row in `Processing` until its alertable `ManualReview` update reaches the writer. If a slow webhook keeps enough updates queued past the five minute recovery threshold and a recovery tick, the following sequence can occur:

1. Recovery verifies that a delayed row has no broadcast release and a clear nonce bit, then changes it from `Processing` to `Pending` with a new `updated_at` value.

2. The fetcher locks the row again. If the mint was reopened, this new incarnation can broadcast and complete its release.
3. The old queued `ManualReview` update reaches the writer first. It matches the broad `processing` predicate and terminalizes the new incarnation.
4. The new `Completed` update is then skipped because the row is already terminal.

The new release signature remains in the pending signature journal, but automatic cleanup cannot use it to repair this row. The status writer supplies no signature arrays when it writes `ManualReview`, while the periodic reconciliation query only selects rows whose transaction record contains both release signatures and their block heights. Boot repair reads the pending journal but treats `ManualReview` as nonrepairable even when one of those signatures landed.

The release has therefore landed while the database still calls the row `ManualReview`. That status remains in the unreleased nonce bounds. Once work reaches a later bitmap generation, the stale lower nonce can prevent bitmap rotation and stop withdrawals for every mint until an operator repairs the row. A continuing alert burst can stop the withdrawal processor earlier by keeping the status channel full.

Recommendation: Move alert delivery to a separate bounded worker so webhook latency cannot delay database status writes. Add the originating `updated_at` value or an immutable attempt identifier to every `TransactionStatusUpdate` and require it in the status update predicate. Only emit an alert after the matching incarnation was updated successfully. This keeps a delayed outcome from changing a row that recovery has requeued and claimed again.

Solana Foundation: Fixed in commit 70c0726.

Cantina: Fix verified. Transaction terminalization now uses a producer-side compare-and-swap keyed by the row incarnation (`updated_at`). Webhooks are alert-only on a bounded queue, and stale completions are dropped. A delayed notification can no longer mutate a recovered/retried transaction instance.

3.2.28 Stale state reads strand reopened withdrawals

Severity: Medium Risk.

Context: processor.rs#L581-L662, processor.rs#L931-L949, recovery.rs#L245-L303, recovery.rs#L347-L425, proof.rs#L168-L235, mint_util.rs#L46-L67, mint_util.rs#L119-L132, mint_util.rs#L182-L210, db.rs#L1989-L2040, db.rs#L2948-L2983.

Description: The withdrawal processor reads the source chain's mutable `AllowedMint` account before building every release. It also reads the target mint's mutable `PausableConfig` when that extension exists. These reads are intended to make an administrator's withdrawal block, reopening, pause or unpause take effect without restarting the operator.

The freshness bound does not track those administrative transitions. After the operator has successfully served a mint once, `read_withdrawal_allowed_mint()` reuses the process lifetime `MintCache::existence_floor` as `minContextSlot`. That slot only proves that the mint existed. It can predate a later `BlockMint` and reopening by an arbitrary interval.

Consider an administrator who blocks withdrawals at slot 800 and reopens them at slot 1000. A running operator may still hold an existence floor of 100. After the reopening is indexed, a user can burn private channel tokens for a new withdrawal. If the configured Solana RPC is load balanced, its account request can reach a lagging backend at slot 900. The backend satisfies `minContextSlot: 100` while returning the old `withdrawals_blocked=true` state. The processor accepts that answer, parks the withdrawal in `ManualReview` and returns before advancing the floor or sending any release transaction.

The same existence floor is passed to the live pause check. A replica snapshot from before an unpause can therefore park a valid withdrawal through the same path.

This differs from the intended manual path for a withdrawal submitted while the gate really was blocked. Here the administrator already reopened the gate before the new burn. The false preflight bail records no release or remint signatures, so periodic reconciliation excludes the row and cannot

clear it automatically. The user's burned balance remains parked until an operator verifies the state and manually returns the row to the queue.

`ManualReview` also remains in the durable unreleased nonce bounds. If a later withdrawal reaches the next bitmap generation, the false lower nonce makes the rotation driver withhold the shared rotation. All higher nonce withdrawals across every mint then remain unavailable until the row is repaired. The trigger requires a block and reopen lifecycle, a long running operator that previously served the mint plus RPC lag during the withdrawal after reopening. A user cannot create those infrastructure conditions on demand.

Recommendation: Before checking any mutable withdrawal gate, obtain a fresh finalized tip and require the `AllowedMint` plus target mint reads to answer at or beyond that slot. Treat an endpoint that cannot satisfy the bound as transient and retry against another backend instead of parking the row.

Solana Foundation: Fixed in commit 0d2967c.

Cantina: Fix verified. Withdrawal decisions bind reads with `minContextSlot` equal to the maximum of the RPC tip, durable escrow checkpoint, and process high-water mark. Related account reads share that bound, and a node unable to satisfy it returns a transient error. Reopened work cannot be terminally judged from stale state.

3.2.29 Stale snapshots can trigger a global halt

Severity: Medium Risk.

Context: get_epoch_schedule_impl.rs#L4-L13, handler.rs#L35-L57, lib.rs#L566-L586, lib.rs#L1243-L1249, escrow_sweep.rs#L236-L259, reconciliation.rs#L299-L348, reconciliation.rs#L496-L514, reconciliation.rs#L547-L600, db.rs#L2824-L2840.

Description: The escrow reconciliation task is intended to stop bridge processing when the PrivateChannel token supply exceeds Solana custody beyond the configured tolerance and the amount of unsettled work. It reads custody from Solana, reads each mint supply from PrivateChannel and loads the unsettled amount from PostgreSQL. Three consecutive breaches create a durable halt.

`fetch_channel_supply()` requests the mint account with finalized commitment but returns only `Mint.supply`. It discards `response.context.slot`. `fetch_escrow_balances_by_mint()` retains the Solana custody context slot, but `perform_reconciliation_check()` uses it only to bound the historical ledger query. It does not require the slot to cover completed source deposits. The PostgreSQL envelope includes `Pending` and `Processing` rows. It also includes `Parked` and `PendingRemint` rows. It stops covering a deposit or withdrawal once that row becomes `Completed`.

This becomes unsafe in both cross chain directions.

For a completed withdrawal in the supported PrivateChannel horizontal deployment:

1. The withdrawal indexer observes a burn through a current replica. The operator releases the corresponding tokens from Solana custody and marks the withdrawal `Completed`.
2. Reconciliation reads the reduced Solana custody. Its channel `getAccountInfo` request is routed through the gateway to a responsive replica that is still before the burn, so it returns the larger supply from before the burn.
3. The completed withdrawal is absent from the envelope. Reconciliation therefore treats the difference between the old supply and current custody as an insolvency breach.
4. If the stale backend serves three consecutive ticks, the third tick writes the durable reconciliation halt.

For a completed deposit through a stale Solana RPC backend:

1. The escrow indexer observes a Solana deposit at source slot `S`. The operator completes the matching PrivateChannel mint and marks the deposit `Completed`.
2. Reconciliation reads current PrivateChannel supply but the configured Solana RPC endpoint returns a finalized custody snapshot from before `S`.

3. The completed deposit is absent from the envelope. The historical ledger is pinned only to the stale custody slot, so it also excludes the deposit at `S`. Nothing proves that the custody snapshot covers the terminal deposit before the supply comparison runs.
4. Three consecutive stale custody snapshots therefore write the same durable halt even though current custody fully backs the current channel supply.

At the default five minute interval, the stale backend must remain behind for longer than ten minutes and serve all three checks. The completed deposit or withdrawal must also exceed the configured tolerance, which defaults to 10 basis points of custody. A bridge user can supply and time that transfer but cannot create the replica lag. The withdrawal direction requires stale PrivateChannel routing after a current replica exposed the burn. The deposit direction instead requires the configured Solana RPC endpoint to keep serving custody from before the already completed source deposit. The resync runbook explicitly recognizes older load balanced account reads and uses a fresh tip plus `minContextSlot` to avoid them.

The repository PrivateChannel readiness path does not remove a responsive stale replica. Gateway readiness only checks whether the read node's `/health` endpoint succeeds. That endpoint calls the static `getEpochSchedule` method and does not compare the replica replay position with the writer. Once either false breach is confirmed, the operator freezes both bridge directions, quarantines every active withdrawal and forces its health endpoint to return an error. It also sends the configured alert, but recovery remains manual even though custody is fully backed.

Recommendation: Bind both cross chain inputs to per mint terminal transition floors before evaluating insolvency. Retain each channel supply response context and require it to be at or after the latest completed withdrawal's PrivateChannel burn slot. Require the Solana custody context to be at or after the latest completed deposit's source slot. Hold the breach counter and retry when either snapshot is below its corresponding floor. Evaluate insolvency only after both snapshots prove that they include every terminal transition excluded from the envelope.

Solana Foundation: Fixed in commit ee63dfc.

Cantina: Fix verified. `fetch_in_flight_envelope` accounts for bridge changes newer than each custody or supply snapshot, so ordinary cross-chain lag no longer creates a false insolvency halt. Because that adjustment can mask a real shortfall, `evaluate_and_maybe_halt` alerts and preserves the breach streak whenever the adjustment flips the verdict; an unadjusted breach still reaches the durable halt after the configured streak.

3.2.30 Unapproved mints block escrow indexer startup

Severity: Medium Risk.

Context: indexer.rs#L543-L552, reconciliation.rs#L215-L225, reconciliation.rs#L317-L350, reconciliation.rs#L389-L516, escrow_sweep.rs#L145-L226, escrow_sweep.rs#L236-L259.

Description: Startup reconciliation is intended to verify the private channel supply of every supported source mint before the escrow indexer begins streaming. `fetch_escrow_balances_by_mint` instead adds the mint from every SPL Token and Token 2022 account whose token authority is the escrow instance. It does not require a corresponding `AllowedMint` account, a positive token balance or an indexed database row. `build_reconciliation_set` consequently treats an unapproved token account as a supported mint.

An ordinary user can select one keypair, use its public key as a private channel System account and create an SPL mint at the same address on the source chain. The Associated Token Account program lets the user pay to create the zero balance account for that mint and the escrow instance without a signature from the instance PDA. On the next escrow indexer startup, the custody sweep includes the selected mint address. `fetch_channel_supply` retrieves the existing private channel System account and fails to decode its empty data as an SPL Mint.

`check_channel_supply_invariant` retains this address as unreadable through all six verification rounds and returns `SupplyInvariantUnverified`. The startup path propagates the error before starting the data source. The source token account is controlled by the instance PDA and the escrow program has no instruction that closes an unapproved account, so the poison remains across restarts.

A user can therefore stop source deposit indexing until the operator deploys code that excludes the account or adds an equivalent recovery mechanism.

This differs from the existing token account closure finding. That issue occupies the private address of a known source mint and prevents its first deposit from completing. Here the user selects an unapproved source mint and uses its instance owned token account to poison the reconciliation universe, which stops the complete escrow indexer after any restart. Reserving known private mint addresses does not remove unapproved mints from this sweep.

Recommendation: Build the reconciliation mint universe from source mints that have a valid `AllowedMint` PDA for the configured instance. Query their expected associated token accounts for custody instead of treating every token account controlled by the instance as supported. Report other instance controlled accounts as anomalies without using their mint addresses for private channel supply verification.

Solana Foundation: Fixed in commit ffadc4b.

Cantina: Fix verified. Startup sweeping now accepts only the canonical `AllowedMint` PDA owned by the escrow program for the configured instance and derives the canonical associated token account. Squatted or unrelated mint accounts are ignored rather than treated as zero-supply obligations that abort startup.

3.2.31 Absent history cursors scan every address row

Severity: Medium Risk.

Context: get_signatures_for_address.rs#L62-L79, get_signatures_for_address.rs#L126-L127, postgres.rs#L204-L213.

Description: `getSignaturesForAddress` limits each history page to 1,000 entries. Before that limit applies, `resolve_cursor()` looks up a caller supplied `before` signature in `address_signatures` using only the `signature` column. The table has a primary key on `(address, slot, signature)` and no index beginning with `signature`. A validly encoded signature that is absent from the ledger therefore makes PostgreSQL scan every row of that table before the request returns an error.

Any user with a verified wallet that owns an existing token account can issue this account gated request. The public gateway permits 50 requests per second and 64 concurrent connections from one IP, while the core PostgreSQL pool defaults to 32 connections. As the ledger grows, repeated absent cursors can occupy that pool and queue ordinary account and transaction reads behind scans that return no history.

Recommendation: Create an index on `address_signatures(signature) INCLUDE (slot)` for `resolve_cursor()`. This keeps the current cursor behavior while making an absent signature an indexed miss.

Solana Foundation: Fixed in commit 13fe3c3.

Cantina: Fix verified. The signature-by-address index now includes slot/cursor ordering suitable for the absent-cursor query and is built under the writer lease. The query plan no longer requires scanning the entire address history when the cursor is missing.

3.2.32 Failed startup fill strands withdrawals

Severity: Medium Risk.

Context: indexer.rs#L788-L817.

Description: On restart, the Withdraw indexer backfills slots above its durable checkpoint while RPC polling begins at the resolved live slot. The startup path arms a checkpoint gate for the missing range, then spawns `BackfillService::run_range` without supervising its result. If the backfill RPC returns errors through the three configured attempts, the task only logs the failure. Polling still starts above that range and never requests its missing slots. The checkpoint gate also cannot advance past the gap, even as later live slots are processed.

The shipped Withdraw indexer enables this backfill path. A withdrawal burned in the failed range gets no pending row for the release operator. Recovery of the RPC endpoint alone does not revisit it. The process continues running until an operator restarts the indexer or repairs the range, leaving the user's withdrawal unreleased in the meantime.

Recommendation: Supervise the startup backfill task and propagate an exhausted fill error to the indexer process. Stop the live source before exit so the configured process restart replays the unresolved range from its durable checkpoint.

Solana Foundation: Fixed in commit 9786acc.

Cantina: Fix verified. Startup backfill is now supervised. If it fails, the service cancels the live source, drains safely without advancing the checkpoint, and exits so restart can retry. It cannot continue indefinitely with a permanent historical gap.

3.2.33 Pings prevent Yellowstone stream recovery

Severity: Medium Risk.

Context: source.rs#L691-L714, source.rs#L792-L803.

Description: The Solana escrow indexer subscribes to finalized Slot updates and escrow Blocks. When a subscription stops delivering chain updates, the datasource is meant to reconnect and fill the missed slots from its durable checkpoint. `connect_and_stream` instead starts a fresh 120-second stall timer around every `stream.next()` call. A subscription `Ping` resets that timer even though it advances no slot and sends nothing to the processor.

If one subscription stops delivering Slots and Blocks while its Geyser connection keeps sending Pings, the stall timer never fires. Even when a fresh subscription can deliver updates, the indexer remains on the old one and does not start gap repair. The supplied Solana escrow deployment uses this Yellowstone source. Deposits made during the fault reach escrow but are not indexed for private credit until useful updates resume or an operator restarts the indexer.

Recommendation: Track the time since the last advancing finalized Slot or Block separately from transport Pings. Reconnect when chain progress stops beyond the configured window, then use the existing gap repair from the durable checkpoint to ingest missed events.

Solana Foundation: Fixed in commit d602bf0.

Cantina: Fix verified. The Yellowstone watchdog records progress only for a newer slot/block; keepalive pings no longer reset it. A stream delivering pings without chain data times out, reconnects, and enters gap repair.

3.2.34 Quiet slots strand withdrawal refunds

Severity: Medium Risk.

Context: source.rs#L630-L665, source.rs#L675-L712, source.rs#L743-L789, remint.rs#L829-L878, remint.rs#L1014-L1056, remint.rs#L1130-L1160.

Description: The withdrawal sender compensates a failed Solana release only after its signatures can no longer land and the escrow indexer has covered the possible payout window. With Yellowstone, only Blocks that match the escrow program filter produce `SlotComplete` messages. Finalized Slot updates keep the stream active but do not advance the escrow checkpoint during a period without matching Blocks.

After a release fails, `release_coverage` reads the current Solana slot and requires the escrow checkpoint to reach that slot before it accepts an absent payout record. This read occurs after the first 32 second remint delay, so the slot can be later than the failed release's Block. If no further escrow instruction occurs, the checkpoint remains below the captured bound despite a healthy finalized stream. Three failed coverage checks move the withdrawal to `ManualReview` and remove it from automatic compensation.

A holder can reach this path by withdrawing to a destination account that rejects the escrow transfer. The holder's private balance has already been burned, no Solana tokens were released and no compensation remint is sent. An operator must resolve the row manually. The `ManualReview` nonce also remains owed for bitmap rotation. If later work reaches the next 65,536 nonce generation, that lower nonce prevents rotation and holds withdrawals for every mint until it is repaired.

Recommendation: Advance a separate finalized coverage watermark through slots proven to contain no escrow instructions and use that watermark for the absent release check. Keep a failed withdrawal eligible for automatic compensation while the activity checkpoint alone trails the captured slot.

Solana Foundation: Fixed in commit be0b82a.

Cantina: Fix verified. `process_pending_reminits` grants bounded free waits for live signatures, pending finality and advancing coverage while retaining the payout and coverage checks before reminting. Yellowstone `connect_and_stream` repairs withheld blocks through RPC. Upstream Yellowstone sends empty filtered blocks, so quiet slots alone do not establish the original stated cause.

3.2.35 Stalled RPC response stops withdrawal indexing

Severity: Medium Risk.

Context: backfill.rs#L473-L501, rpc.rs#L25-L45, rpc.rs#L357-L375, source.rs#L139-L170.

Description: The withdrawal indexer polls the private channel for `WithdrawFunds` instructions and stores them as pending rows for the release operator. `RpcPoller::new` creates an HTTP client without a request deadline. The source awaits `getSlot` before spawning when it has no resume slot and inside its single live polling loop. The enabled startup backfill uses the same client to read the tip. A response that sends its headers but withholds the remaining body leaves that future pending, so the configured error retry interval never starts. A fresh request to the same recovered read node can succeed while the old polling request still holds the indexer.

The shipped withdrawal indexer configurations use RPC polling against the read node. When one of its requests stalls, later burns are not indexed and the release operator has no new pending rows to process. The live source cannot check cancellation until the request returns. A stall before the first completed slot also leaves the indexer's health state without a progress timestamp, which its health check treats as healthy. Users who burn during the stall wait for an operator restart or the old response to close before their withdrawals can be released.

Recommendation: Set a configurable deadline on the complete `RpcPoller` exchange, including response body delivery. Treat expiry as a transport error so polling and backfill retry from the same uncheckpointed slot. Allow datasource cancellation to interrupt a pending request.

Solana Foundation: Fixed in commit 003983b.

Cantina: Fix verified. Every `RpcPoller` request is wrapped in a fixed full-exchange deadline, and cancellation is raced against outstanding reads. Timeout or cancellation returns without checkpointing, allowing the same slot to retry rather than hanging the worker forever.

3.2.36 Zero-value deposits create operator-funded channel state

Severity: Medium Risk.

Context: deposit.rs#L127-L158.

Description: A deposit should carry enough value to justify the persistent source row and private-channel work it creates. The escrow program instead accepts an `amount` of zero. Unlike the withdrawal program, it performs no zero-amount check before invoking `TransferChecked`. SPL Token accepts a zero transfer between valid accounts, so the escrow balance remains unchanged and the program emits a successful `DepositEvent` whose received amount is zero.

The indexer persists every such event as a distinct pending deposit. The operator then builds a normal private-channel mint transaction for the row. That transaction first creates the recipient's associated token account idempotently with the channel admin as payer, writes an idempotency memo, and

finally mints zero tokens. A caller can choose an arbitrary recipient for every deposit, so the ATA creation is not naturally deduplicated. For a source mint that has not yet been used on the channel, the first zero deposit also drives the JIT mint-initialization path.

An ordinary user only needs a valid empty source ATA for an allowed mint. They can submit repeated zero deposits, including several instructions in one source transaction with different recipients. No source token balance is consumed, but every instruction creates an indexer row, operator work, a channel transaction and address-index history, and potentially a permanent recipient ATA paid for by the service identity. Sustained submissions can grow both databases and occupy the serial deposit pipeline ahead of value-bearing deposits.

Recommendation: Reject a deposit whose requested amount is zero before any CPI. Also reject a deposit when the measured `received` amount is zero, since Token-2022 fees or future token behavior can reduce a positive request to no received value. Add the same guard to the client builder and indexer as defense in depth, and set a per-mint minimum deposit that covers the destination work and persistent state the bridge sponsors.

Solana Foundation: Fixed in commit e1ce643.

Cantina: Fix verified. The escrow program rejects both a requested zero amount and a transfer whose measured net receipt is zero. Minimum-deposit enforcement uses net receipt after token fees, and the indexer defensively ignores any zero-valued event. Zero deposits can no longer trigger operator-funded channel state.

3.2.37 Operational scripts expose secrets in process arguments

Severity: Medium Risk.

Context: update-admin-env.sh#L23-L30.

Description: Operational secrets should not be copied into command-line argument vectors, which are commonly visible to other local users, process monitors, crash collectors and CI telemetry. Two shipped helpers do so.

`update-admin-env.sh` reads the complete channel-admin keypair into `admin_private_key` and passes the JSON byte array as the fourth argument of a child `upsert-env.sh` process. During that process lifetime, the full mint-authority key is present in its argv rather than only in the protected source keypair or destination file.

`reconcile-escrow-balance.sh` requires a PostgreSQL connection URL as its third positional argument, and its own example embeds `user:pass` in that URL. The secret therefore remains in the reconciliation process's argv for the whole RPC and database workflow. The script then passes the same URL as an argument to `psql`, creating another observable argv copy.

On a conventional multi-user host, a local account or monitoring agent that can inspect process command lines can recover the channel admin private key when the environment helper runs, or the indexer database password while reconciliation runs. The admin key can authorize private-channel mints and remints; the database credential can expose or mutate bridge state according to its grants. Gitignore rules do not protect either transient argv copy. Recovery requires rotating the exposed credential, not merely deleting shell history.

Recommendation: Never pass secret values as positional arguments. Have `update-admin-env.sh` write the key inside the same process through a protected file descriptor or stdin, with the destination helper accepting the value on stdin. For PostgreSQL, accept a non-secret service name or connection parameters and use a mode-0600 `PGPASSFILE`, `.pgpass`, or a secret manager rather than a password-bearing URL argument. Add tests that inspect spawned command lines and fail if known secret material appears.

Solana Foundation: Fixed in commit 2b96388.

Cantina: Fix verified. Operational scripts now pass secrets through standard input, environment variables, libpq password files, or `curl -K -` input rather than command-line arguments. The secret is no longer exposed in ordinary process listings.

3.2.38 Admin env helper creates world-readable key files

Severity: Medium Risk.

Context: upsert-env.sh#L14-L36.

Description: `update-admin-env.sh` promises to keep the channel admin private key in a gitignored runtime env file, but the generic helper it calls does not establish secret file permissions. When the destination does not exist, `upsert-env.sh` creates it with a normal shell redirection. Under the common process umask `022`, the resulting mode is `0644`, so every local user that can traverse the directory can read `ADMIN_PRIVATE_KEY`.

The append path has the same problem for a pre-existing permissive file: it preserves that file's mode and appends the secret. Only the replace-existing-key path happens to move a `mktemp` file over the destination, and the code neither relies on nor verifies its owner or mode. The regression test checks that the key is absent from the tracked template, but never checks confidentiality of the runtime file.

A safe local reproduction is:

```
d="$(mktemp -d)"
( umask 022; ./scripts/upsert-env.sh "$d/runtime.env" ADMIN_PRIVATE_KEY test-secret
↪  )
stat -f '%Lp %N' "$d/runtime.env"
# 644 .../runtime.env
```

In the intended repository or deployment directory, which is normally traversable by other host users, the generated `.env` exposes the full channel mint-authority key. A local account, backup agent or unrelated service identity can copy it and mint unbacked channel tokens. Removing the file after compromise is insufficient; the key must be rotated.

Recommendation: Set `umask 077` before handling the private key and create the destination atomically with mode `0600` in its final directory. Refuse symlinks and files not owned by the invoking user, and explicitly `chmod 0600` both newly created and existing runtime files before writing. Extend `update-admin-env.test.sh` to assert the owner and exact mode on create, append and replace paths.

Solana Foundation: Fixed in commit 47dc513.

Cantina: Fix verified. The helper sets `umask 077`, refuses symlinks, nonregular targets, and foreign ownership, creates same-directory mode-0600 temporary files, and atomically renames them. Key material is never intentionally installed with group/world access and unsafe targets fail closed.

3.2.39 Transfer fees can erase withdrawal payouts

Severity: Medium Risk.

Context: processor.rs#L665-L808, allow_mint.rs#L68-L99, release_funds.rs#L108-L177, token_-utils.rs#L138-L175, withdraw_funds.rs#L30-L66.

Description: The supported asset policy excludes withdrawals for Token-2022 mints that carry `TransferFeeConfig`. The escrow does not enforce that restriction. `read_mint_profile()` accepts every extension and `process_allow_mint()` records the resulting profile without rejecting a transfer fee mint. Deposits remain solvent because `process_deposit()` credits the measured increase in escrow custody after the fee.

The withdrawal path does not preserve the same user value. `process_withdraw_funds()` burns the requested private amount and the operator preflight checks pause, freeze and permanent delegate state but never checks `TransferFeeConfig`. `process_release_funds()` then executes `TransferChecked` for the gross burned amount. It records the gross decrease in escrow custody and consumes the withdrawal nonce without checking how much spendable balance reached the recipient.

After a fee mint has been allowed, its fee configuration authority can raise the fee before a user withdraws. At a 100% fee, a release of 100,000 units succeeds and removes 100,000 units from escrow.

The recipient's spendable balance does not increase. The complete amount becomes withheld fees that the configured withdrawal authority can collect and the nonce remains consumed. Because the release succeeded, the compensation path does not restore the user's burned private balance.

This affects users of an allowed fee mint after the issuer changes its fee. The trusted allowlist action is a prerequisite, but the resulting payout contradicts the explicit supported asset policy and lets the fee authorities confiscate a completed withdrawal.

Recommendation: Reject `TransferFeeConfig` in `process_allow_mint()` and refuse new deposits for any existing allowlist entry that carries this extension. The withdrawal operator should also detect the extension before accepting another private burn. Provide a separate administrator controlled unwind for balances that were already credited so blocking the unsupported payout path does not strand those users.

Solana Foundation: Acknowledged. Allowlisting a mint is an explicit issuer-trust decision, documented in the Mint Risk Considerations table (docs/ESCROW_INTERACTION_GUIDE.md). The TransferFeeConfig row states that the fee falls on the user on withdrawal, that the fee authority can raise it after deposit, and that neither the program nor the preflight caps it. Escrow solvency is unaffected: both legs account by measured balance delta.

3.2.40 Unavailable bitmap validation lets the withdrawal operator start unsafely

Severity: Medium Risk.

Context: operator.rs#L539-L570.

Description: The withdrawal startup preflight compares withdrawals recorded as `Completed` with the current on-chain withdrawal bitmap. A completed database row whose nonce is clear on-chain means the database claims a payout that the escrow never made, so `validate_bitmap_consistency()` correctly classifies that state as `BitmapDivergence` and refuses startup.

The caller only refuses startup when it receives that exact divergence error. If the bitmap account, finalized anchor, or database cannot be read, `run_withdraw_preflight()` logs the error and starts every withdrawal worker anyway. Its comment says that the recovery worker will revalidate the bitmap, but the production recovery loop calls `recover_once()` and never invokes `validate_bitmap_consistency()`; outside tests, the validation function is called only by the startup preflight.

Consequently, a transient RPC or database failure during restart can permanently skip the sole check for an already inconsistent completed withdrawal. The completed row is not fetched again as owed work and is excluded from the unreleased nonce bounds. Once later withdrawals require the bitmap to rotate, the operator can advance past that generation without paying the missing release, destroying the current-generation evidence that would have exposed the inconsistency.

This does not let an ordinary user manufacture the initial divergence by itself, but it turns a recoverable consistency failure, such as a previously orphaned release, into continued unsafe operation whenever the validation dependencies are unavailable at boot.

Recommendation: Treat an unavailable bitmap verdict as a startup failure, not as permission to process withdrawals. Retry the complete check with bounded backoff before starting the fetcher, sender, or rotation worker. If process availability requires the service to bind first, keep health and readiness failed and hold a durable withdrawal-processing freeze until the check succeeds. If continuous validation is intended, explicitly run it from a periodic worker and prevent claims and bitmap rotation whenever its most recent verdict is unavailable.

Solana Foundation: Fixed in commit 365bdbe.

Cantina: Fix verified. Withdrawal startup now performs a bitmap preflight and fails closed on every non-verdict. A missing generation-zero bitmap is accepted only when no completed releases exist; completed releases above or inconsistent with the observed generation prevent worker startup. Replay protection is established before release processing begins.

3.2.41 CPI deposits without stack heights permanently block replay

Severity: Medium Risk.

Context: escrow.rs#L532-L558.

Description: The escrow program supports being called through CPI, and the indexer uses each inner instruction's optional `stackHeight` to associate a CPI `Deposit` with the `DepositEvent` emitted from its subtree. The RPC and Yellowstone types both represent this field as optional because historical RPC metadata and some Geyser feeds can omit it.

When a CPI deposit has no stack height, `parse_deposit()` unconditionally returns `Instruction ParseFailed`. It does so even when the transaction contains exactly one CPI deposit and one fully decodable escrow deposit event, where no competing attribution exists. The parser does not attempt to bind the event's instance, user, recipient, mint, and amount to the instruction accounts.

The decoder treats any parsing failure for a supported instruction as uncertainty about the entire slot. Live polling therefore refuses to checkpoint the slot and retries it indefinitely. Startup backfill tries one fallback endpoint and then aborts before advancing the checkpoint. If a destructive resync encounters the record, the rebuild fails only after the old tables have already been dropped and recreated.

A legitimate application that wraps `Deposit` in CPI can therefore create a permanent replay barrier whenever the selected history source omits stack heights. All later deposits and withdrawals remain behind that slot until the data source changes or parser compatibility is deployed. The transaction itself succeeded on-chain, so retrying the same historical endpoint cannot repair the metadata.

Recommendation: Keep stack height as the preferred way to isolate nested call trees, but provide a conservative compatibility path when it is absent. Decode the complete `DepositEvent` and accept it only when exactly one candidate matches the deposit instruction's instance, user, recipient, mint, and expected account relationships; reject only genuinely ambiguous transactions. Add fixtures for historical RPC and Yellowstone responses with null stack heights. Before destructive resync, also pre-flight the selected archival endpoints across the rebuild range so incompatible metadata is detected before tables are dropped.

Solana Foundation: Fixed in commit e4b2909.

Cantina: Fix verified. Deposit parsing uses `stackHeight` when available; otherwise it selects the first subsequent escrow event entry and validates user, mint, recipient, and instance. Ambiguous or inconsistent fallbacks fail the slot instead of checkpointing it. Supported CPI deposits can be replayed without misassociation.

3.3 Low Risk

3.3.1 Fee monitor watches an unrelated wallet

Severity: Low Risk.

Context: feepayer_monitor.rs#L20, instruction_util.rs#L199-L203.

Description: `ReleaseFunds` places the admin signer first and the sender always uses the first signer as the transaction fee payer. These transactions are created by the Withdraw operator against its Solana destination RPC. The fee monitor is instead started only by the Escrow operator against its source RPC and it always queries the operator signer balance.

When admin and operator are distinct keys, the reported metric and low balance warning describe a wallet that does not pay release fees. User withdrawals or routine retries can therefore deplete the actual admin fee payer without this monitor reporting the condition. Release transactions then stop until the admin wallet is funded, despite the monitored operator wallet having sufficient SOL.

Recommendation: Run the release fee monitor in the Withdraw role against its destination RPC and query the admin public key that the builder selects as fee payer. Derive the monitored address from the same fee payer selection used to build transactions so later signer ordering changes cannot make telemetry drift again.

Solana Foundation: Fixed in commit 471e2e4.

Cantina: Fix verified. The fee-payer monitor now runs only for the Withdraw operator, reads the destination `rpc_url` used for release broadcasts, and watches `TransactionBuilder::release_signers()[0]`, the signer the sender bills for releases and rotations. Only that monitor seeds the balance metric, and its alerts and dashboards select the Withdraw operator, restoring monitoring of the key and chain whose depletion can stop withdrawals.

3.3.2 Privy signer configuration panics at startup

Severity: Low Risk.

Context: signer_util.rs#L235-L237.

Description: Privy is a documented signer backend for both the admin and operator. The indexer enters its Tokio runtime and then calls the synchronous `OperatorConfig::validate_signers` function. When lazy initialization reaches the Privy branch, `load_signer` invokes `Handle::current().block_on` for the asynchronous `Signer::from_privy` constructor.

Tokio panics when `block_on` starts a runtime operation from a thread that is already driving the runtime. As a result, any operator configured with complete Privy credentials terminates during signer validation before its bridge workers start. The affected bridge role stays unavailable until the signer configuration or implementation changes.

Recommendation: Make signer initialization asynchronous and await `Signer::from_privy` from the existing runtime. Store the result in an asynchronous once cell so every caller receives either the initialized signer or a normal configuration error without starting a nested runtime.

Solana Foundation: Fixed in commit ea80bdf.

Cantina: Fix verified. `build` now awaits `Signer::from_privy` on the existing runtime, while `init_signers` initializes and caches both signer roles before binary and library operator startup and `init_admin_signer` does the same for resync. The synchronous accessors no longer invoke `block_on`, so valid Privy configuration cannot trigger the nested-runtime panic, configuration failures return startup errors, and a set but invalid operator signer cannot silently fall back to the administrator.

3.3.3 Storage errors produce a fake blockhash

Severity: Low Risk.

Context: get_recent_blockhash_impl.rs#L10-L26.

Description: `getRecentBlockhash` is intended to return a usable recent hash and its context slot. The handler converts a failed blockhash lookup to `Hash::default()` and independently converts a failed slot lookup to zero. It then returns both values in a successful JSON RPC response.

The gateway routes this read to the read node while it routes `sendTransaction` to the write node. A read store failure can therefore give a client the all zero hash while the write path remains available. The client signs with that nonexistent hash and `sendTransaction` returns its signature after enqueueing it. The dedup stage later drops the transaction because the hash is outside its live window. This leaves the client polling a signature that cannot appear even though the blockhash request reported success.

Recommendation: Propagate either storage failure as a JSON RPC server error. Only return a blockhash response after both the hash and its context slot have been loaded successfully from the same available ledger view.

Solana Foundation: Fixed in commit 00657a3.

Cantina: Fix verified. `get_recent_blockhash_impl` now calls `get_blockhash_snapshot` once and maps snapshot failures to a JSON-RPC server error, so a failed, absent, or malformed hash cannot become a successful all-zero value. PostgreSQL reads the context and hash in one statement; Redis accepts only a current, complete `MGET` result, falls back to the complete PostgreSQL snapshot for

missing, stale, or unreadable cache data, and rejects an invalid cached hash. The atomic tip writers preserve that grouping, restoring the invariant that every successful response carries a stored hash and context from one ledger view.

3.3.4 Unused writable keys serialize channel execution

Severity: Low Risk.

Context: send_transaction_impl.rs#L42-L102, dag.rs#L40-L87, dag.rs#L92-L177, types.rs#L43-L54, execution.rs#L1252-L1304.

Description: The public `sendTransaction` endpoint restricts which programs a transaction can execute. Its admission path validates the message's account locks and checks every top level instruction against the program allowlist. It does not require writable message accounts to be referenced by an instruction.

A signed Solana message can therefore declare a known channel account writable while no instruction receives that account. For example, an attacker can submit an SPL Memo instruction with no instruction accounts while the message declares the channel mint as an unsigned writable key. The transaction passes public admission and signature verification. Distinct payers and Memo data give every carrier a distinct message hash, so ordinary deduplication retains the transactions.

`extract_accounts()` derives scheduler locks from the complete sanitized message. The dependency graph consequently treats the unused channel mint as a real write target. Every attacker carrier conflicts with every other carrier and with an honest MintTo that uses the same mint. In the default 64 transaction collection, 64 such carriers become 64 one transaction batches. A mixed collection containing 63 carriers followed by one honest MintTo also produces 64 batches and places the honest transaction last.

Each carrier executes successfully while leaving the selected account byte for byte unchanged. The one transaction batches also stay below the executor's parallel threshold. A native local debug comparison using the default eight workers took 151 milliseconds for the 64 conflicting carriers and 14 milliseconds for 64 carriers with distinct unused keys, a 10.29 times difference. This timing measures one test host and is not a production throughput benchmark.

The gateway exposes `sendTransaction` without authentication. Its default public limit is 50 requests per second per IP with a burst of 100, so one source is bounded while distributed sources remain possible. Sustained carriers can consume write execution capacity, delay bridge transactions that use the chosen account and plausibly fill the bounded ingress queue until honest submissions receive the retryable capacity error. The proof does not measure an end to end outage or establish a fixed production throughput ceiling.

Recommendation: At RPC admission, reject any writable message key other than the fee payer when no top level instruction includes that key in its account indices. Programs invoked through CPI can only receive accounts supplied to their outer instruction, so this check preserves valid CPI account forwarding.

Solana Foundation: Fixed in commit 25235573.

Cantina: Partially fixed. `send_transaction_impl` and `simulate_transaction` now reject writable non-payer keys absent from every non-Memo top-level instruction, as well as writable program and reserved keys, so the reported zero-account Memo carrier is blocked. However, `has_unused_writable_account` treats any key listed by a non-Memo instruction as used. A caller can append a chosen hot writable key to SPL Token `AmountToUiAmount`; the token program reads only its mint and ignores trailing accounts, while `extract_accounts` still schedules the hot key as a writer. These transactions execute successfully without changing the hot key, so the same cross-transaction serialization remains reachable.

3.3.5 Zero allowance delegates can read private state

Severity: Low Risk.

Context: auth.rs#L99-L103, auth.rs#L524-L572.

Description: The gateway allows a token account delegate to call `getAccountInfo` and `getTokenAccountBalance` because a current delegate can spend part of the account balance. This access is meant to follow the delegate's allowance.

`check_token_account_ownership()` checks whether the account has `delegate = Some(pubkey)` and whether that pubkey belongs to the authenticated user. It does not read the `delegated_amount` field at bytes 121 through 128 of the SPL Token base account. The SPL Token `Approve` instruction accepts an amount of zero and writes `delegate = Some(pubkey)` together with `delegated_amount = 0`. Only the separate `Revoke` instruction clears the delegate field.

An account owner can therefore remove all spend allowance with `Approve(delegate, 0)` while leaving the delegate pubkey present. The delegate cannot transfer or burn one token unit, but a gateway user with that verified wallet still passes the ownership check. The gateway returns the account's current amount and full account bytes. This access persists across later balance changes until the owner sends `Revoke`, approves another delegate or changes the account owner.

The trigger requires an owner to approve zero rather than send `Revoke` and the reader must be a previously trusted delegate with a valid gateway account. Even with those conditions, the reader has no current spend authority and can continue observing private state for an unbounded period.

Recommendation: Decode the complete SPL Token base account before granting delegate access. Accept a delegate for current state methods only when the account is initialized, the delegate field matches a verified wallet and `delegated_amount` is greater than zero. Keep the owner check independent from the delegate allowance.

Solana Foundation: Fixed in commit 25235573.

Cantina: Fix verified. `check_account_data_ownership` routes both token programs into `check_token_account_ownership`, which now requires a present matching delegate, a non-Uninitialized account state, and nonzero `delegated_amount` before `getAccountInfo` or `getTokenAccountBalance` proceeds. `Approve(delegate, 0)` therefore fails authorization, while the owner lookup still runs first and remains independent of the delegate allowance.

3.3.6 Full resync can exhaust indexer memory

Severity: Low Risk.

Context: backfill.rs#L54-L66, backfill.rs#L171-L190.

Description: A destructive resync rebuilds the indexer database from a chosen source slot in fixed size batches. `calculate_batches` instead creates a `Vec<Vec<u64>>` that contains every slot in the full range before `fill_slot_range` performs its first RPC request. A positive `batch_size` controls the size of each inner vector but does not bound the total slots retained in memory.

The resync command defaults its genesis slot to zero and removes the normal gap limit. It drops and recreates the database tables before entering the backfill path. With the shipped batch size of 100, a ten million slot range used about 97 MB of resident memory solely for this batch list. Memory grows in proportion to the complete range, so a sufficiently long history can terminate the process after its prior index has been removed.

A trusted administrator must explicitly authorize and run the destructive resync command. An ordinary user cannot select the range. The failed recovery extends indexing and bridge downtime until the administrator chooses a later valid floor or provisions more memory and runs the command again.

Recommendation: Generate and process one slot batch at a time in `fill_slot_range` so resident batch memory remains bounded by `batch_size` for startup, reconnect and resync fills.

Solana Foundation: Fixed in commit 00657a3.

Cantina: Fix verified. `calculate_batches` now yields one slot vector lazily, and `fill_slot_range` consumes it before advancing; `BackfillService::run_range` routes startup and resync through

that loop, while `fill_reconnect_gap_to` routes Yellowstone gap repair through it. Production configuration caps each batch at 100 slots, so retained slot-list memory is independent of the total range, while ascending coverage, empty ranges, and the `u64::MAX` boundary remain correct.

3.3.7 Withdraw startup permits unusable configuration

Severity: Low Risk.

Context: operator.rs#L93-L127, operator.rs#L156-L190, processor.rs#L407-L414.

Description: The Withdraw operator needs an escrow instance ID to check the withdrawal bitmap at startup and build `ReleaseFunds` instructions. The supplied operator example says to omit `escrow_instance_id` for the Withdraw role. When that configuration is used, `operator::run` logs a warning, skips its bitmap preflight and starts the workers. The processor then exits immediately because it has no instance ID. Task supervision shuts the operator down.

Users can still burn private channel tokens while this role is unavailable. Their indexed withdrawals receive no Solana release until the operator is reconfigured and restarted. The packaged local Withdraw configuration includes the instance ID, so this affects deployments that omit it as directed by the example.

The same startup path also accepts a blank `source_rpc_url` as present. With a valid instance ID, ordinary releases can proceed, but a failed release cannot be compensated: the source RPC client rejects the empty URL before the remint can obtain a blockhash. Three deferred attempts move the burned withdrawal to `ManualReview`. The packaged local configuration supplies a source URL, so this variant requires a blank override.

Recommendation: Validate all required Withdraw inputs before starting its workers. Require an `escrow_instance_id` and a nonblank `source_rpc_url` for the compensation chain. Correct the example to show the required instance ID.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `validate_operator_startup` now requires an escrow instance and a nonblank source RPC. `build_operator_config` validates before opening storage and `operator::run` repeats the checks before acquiring the live lock. Both pass the trimmed source URL to the workers. The example now documents the required inputs.

3.3.8 Parsed RPC encoding halts block ingestion

Severity: Low Risk.

Context: rpc.rs#L25-L45, rpc.rs#L50-L96, types.rs#L29-L46.

Description: The RPC poller exposes `encoding` as the complete `UiTransactionEncoding` enum and copies the selected value into every `getBlock` request. Its response decoder does not vary with that selection. `RpcBlock` always requires `accountKeys` to contain strings and requires instructions to use the compiled instruction shape expected from `json`.

A trusted deployer can set `encoding` to `jsonParsed` through the TOML configuration or `INDEXER_RPC_POLLING_ENCODING`. The configuration parser accepts that standard Solana variant. A valid parsed block represents each account key as an object, so `get_block_present()` returns a deserialization error for every transaction bearing block.

`RpcPollingSource` handles the error by breaking before it emits `SlotComplete` or advances `current_slot`. It then requests the same slot again. The startup backfill, Yellowstone gap repair and destructive resync use the same poller and cannot cross the block either. Deposits or withdrawals after the blocked slot remain unindexed until the operator corrects the setting and restarts the process. The default and shipped configurations select `json`, which limits this issue to a trusted configuration error.

Recommendation: Reject every `RpcPollingConfig.encoding` value other than `UiTransactionEncoding::Json` before starting an ingestion or resync worker. Expose another variant only after the poller decodes its corresponding standard response shape.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `validate_rpc_encoding` accepts only `Json`. `IndexerConfig::validate` and `indexer::run` enforce it before storage opens, including the configuration used by Yellowstone gap repair. `validate_resync_config` applies the same rule before resync can alter stored data.

3.3.9 Transient journal read parks paid withdrawal

Severity: Low Risk.

Context: state.rs#L372-L400, state.rs#L483-L552.

Description: At startup, the withdraw operator compares consumed bitmap nonces with withdrawals recorded as `Completed`. For a consumed nonce whose row is still `Processing`, it reads the persisted release signatures to identify a landed payout. `load_persisted_release_signatures` converts a storage read error into an empty signature list. `resolve_chain_ahead_nonce` then treats the payout as unattributed and sends a `ManualReview` status update.

The bounded recovery sweep before this check can miss the affected row. If 100 older `Processing` withdrawals have signatures that remain live, each of the eight recovery passes reads that same oldest batch. A newer withdrawal whose release landed then reaches bitmap validation while still `Processing`. One transient error reading its signature journal moves that paid withdrawal to `ManualReview` even though the signature was persisted.

A later healthy journal read does not automatically correct the row. Bitmap repair only completes rows still in `Processing` or `PendingRemint`, while periodic reconciliation requires signature arrays on the transaction row that this status update does not write. The paid withdrawal remains incorrectly unsettled until an operator repairs it. Its nonce also remains owed, so it can prevent bitmap rotation when withdrawals reach a later generation.

Recommendation: Preserve a storage read error as a distinct result from an empty signature journal. Defer the status update for that nonce until the journal can be read again. Keep the withdrawal in `Processing` during that retry so neither completion nor a new release is inferred without evidence.

Solana Foundation: Fixed in PR 384, merged as `00657a3`, with the journal handling introduced in `44e58b4` and finalized in `5a9968f`.

Cantina: Fix verified. `load_pending_sigs` now preserves an unavailable journal as an error after bounded retries. At boot, `resolve_chain_ahead_nonce` leaves the withdrawal in `Processing` without a status write, while `validate_bitmap_consistency` continues checking the other consumed nonces so the read error cannot hide a double payout. The cursor-based release-promotion worker retries journaled `Processing` rows and completes the paid withdrawal once its finalized signature is readable. `handle_nonce_already_used` applies the same leave-for-recovery behavior on the runtime path, so neither affected caller converts a transient read failure into an empty journal or a terminal status.

3.3.10 Unsafe connection caps defeat IP isolation

Severity: Low Risk.

Context: config.rs#L56-L64, config.rs#L107-L130, serve.rs#L139-L143, serve.rs#L195-L218, lib.rs#L119-L126, lib.rs#L1377-L1449, lib.rs#L1493-L1557.

Description: The gateway and auth listeners use a global connection cap plus a smaller cap for each client IP. The smaller cap is intended to keep one source from consuming all global slots. Their safe defaults are 1,024 global connections and 64 connections per IP but both values are configurable.

The gateway accepts any nonzero pair and copies it into `Limits` without checking their relationship. Auth has a relationship check but rejects only values where `max_connections_per_ip` is greater than `max_connections`. It accepts equality even though the validation error and listener comments state that one client must not be able to take the whole budget.

Both listeners acquire the global permit first and then the per IP slot. They retain both for the lifetime of the HTTP connection. When the per IP value equals or exceeds the global value, one remote address can therefore open every permitted connection. The caller can retain them with cheap `/health` requests on HTTP keep alive because the header deadline starts again after each response. New connections from every other address are then dropped at the global limit, including gateway transaction submissions and auth login requests.

This affects deployments where a trusted operator lowers the global cap to or below the per IP value or raises the per IP value to that level. The shipped defaults are safe, which limits the likelihood, but both services accept the unsafe configuration and start normally.

Recommendation: Reject startup unless `max_connections_per_ip` is strictly less than `max_connections` in both services. Apply the same relationship check to the gateway and correct the auth comparison so every accepted configuration reserves global capacity for another client IP.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. Auth `Config::validate` and gateway `check_limits` require the per IP cap to be strictly below the global cap before opening database connections. The listeners receive these checked values, so every accepted configuration reserves at least one global slot for another client IP.

3.3.11 Whitespace JWT secret disables gateway RBAC

Severity: Low Risk.

Context: config.rs#L24-L25, config.rs#L107-L130, lib.rs#L416-L420, lib.rs#L524-L527, lib.rs#L818-L825.

Description: The auth service and public gateway use `JWT_SECRET` as their shared decision that role based access control is enabled. The auth configuration accepts any string and `JwtConfig::new()` signs tokens with its raw bytes. Its `validate()` method does not reject an empty or whitespace value.

The gateway interprets this setting differently. `configured_secret()` trims the value when it decides whether the secret is present. A whitespace value therefore becomes `None`. `Gateway::new()` builds no decoding key and `enforce_auth()` returns a pass through policy for every request when that key is absent.

This affects a deployment whose secret manager or environment rendering supplies a whitespace value while starting the optional auth service. The auth process remains active and issues tokens but the public gateway treats authentication as disabled. An anonymous caller can then invoke operator methods such as `getBlock` plus `getTransaction` plus `simulateTransaction`. The caller can also read token account state and history that normally require a verified wallet. The gateway logs that enforcement is disabled at startup and the condition requires a trusted configuration error, which limits its likelihood.

Recommendation: Use one explicit authentication mode in both services. When that mode is enabled, reject `JWT_SECRET` if its trimmed value is empty and require the auth database before either service begins listening. When the mode is disabled, do not start the auth issuer. Keep the raw nonempty secret bytes unchanged for signing and verification so both services use the same key.

Solana Foundation: Fixed in PR 375 (`464ca71`). Both services now require a `JWT_SECRET` of at least 32 bytes after trimming while retaining the raw bytes for signing and verification. Only an absent or exactly empty gateway value disables authentication; auth rejects an empty or invalid value, and an enabled gateway also requires `AUTH_DATABASE_URL`.

Cantina: Fix verified. `Config::validate()` rejects empty, whitespace-only, and short secrets before auth connects or binds. Gateway `run()` invokes `resolve_jwt_secret()` before database or listener

setup, rejects a whitespace-only value under the same rule, and requires `AUTH_DATABASE_URL` when authentication is enabled; `JwtConfig` and `Gateway::new()` then use the accepted raw bytes. The only production `Gateway::new()` call is behind `run()`, restoring the invariant that the issuer cannot run while the public gateway interprets the same secret as RBAC-disabled.

3.3.12 Zero backfill batch size exhausts the indexer

Severity: Low Risk.

Context: backfill.rs#L54-L66, backfill.rs#L171-L190.

Description: The indexer divides each startup or reconnect backfill range into batches before requesting any blocks. The trusted deployer supplies `backfill.batch_size` as an unrestricted `usize` and `IndexerConfig::validate` does not reject zero. For a positive range and a zero batch size, `calculate_batches` sets `batch_end` to the current `next_slot`, appends an empty batch and assigns the same value back to `next_slot`. The synchronous loop therefore keeps allocating empty vectors without reaching an awaited operation. It cannot return an error or observe cancellation.

A startup gap or Yellowstone reconnect gap under this configuration exhausts the indexer process memory. Deposits or withdrawals in the missing range remain unindexed until the operator corrects the configuration and restarts the process. This condition requires a trusted operator to set the backfill batch size to zero. An untrusted user cannot select the value.

Recommendation: Reject a zero `backfill.batch_size` in `IndexerConfig::validate` before any ingestion worker starts.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `validate_rpc_batch_size` now rejects zero on active ingestion, backfill and resync paths before storage opens. `calculate_batches` also floors direct inputs at one, so each iteration advances even when a caller bypasses startup validation.

3.3.13 Zero retry budget quarantines withdrawals

Severity: Low Risk.

Context: indexer.rs#L87-L105, indexer.rs#L216-L223, indexer.rs#L440-L460, recovery.rs#L245-L266, recovery.rs#L290-L304, transaction.rs#L542-L560, transaction.rs#L1586-L1634, db.rs#L1989-L2060.

Description: The Withdraw operator uses `retry_max_attempts` to bound repeated `ReleaseFunds` submissions. `OperatorSection` accepts this setting as an unrestricted `u32`, environment values can override the TOML value and operator startup does not reject zero.

Each release begins with zero recorded attempts. With a zero budget, the first call to `send_and_confirm` satisfies `attempts >= state.retry_max_attempts` before `build_and_sign` runs. It calls `handle_permanent_failure` without creating or broadcasting a transaction. The withdrawal has no signature journal, so the failure handler emits a `ManualReview` update instead of queuing a compensating remint. The periodic stalled withdrawal reconciliation selects only rows with at least one recorded release signature. As a result, correcting the setting and restarting the operator does not automatically recover these rows.

While this configuration is active, users can continue burning private channel tokens. Every indexed withdrawal that the operator claims is quarantined without a Solana payout and requires manual repair. A trusted deployer must set the value to zero. The packaged configurations set it to three and an untrusted user cannot change it.

Recommendation: Reject `retry_max_attempts` values below one before the operator initializes storage or starts any worker.

Solana Foundation: Fixed in PR 375 (`464ca71`). `validate_operator_startup` rejects `retry_max_attempts = 0`; `build_operator_config` runs that check after TOML and `OPERATOR_RETRY_MAX_`

`ATTEMPTS` are merged and before the database pool opens, while `operator::run` repeats it before the live lock and schema migration for library callers. The withdrawal manual-review runbook adds a guarded recovery path for rows quarantined before the check existed.

Cantina: Fix verified. Both operator startup paths now reject a zero retry budget before storage or workers are touched, restoring the invariant that the retry ceiling cannot fire before at least one `ReleaseFunds` sender attempt enters transaction construction. The environment override reaches the same validation, and the guard remains present through `5e1448b`. Legacy rows require the documented on-chain non-landing check before they are re-armed, which avoids an unsafe automatic replay.

3.3.14 Zero verifier count stops the write node

Severity: Low Risk.

Context: node.rs#L41-L43, node.rs#L197-L202, node.rs#L226-L238, node.rs#L279-L293, node.rs#L210-L237, node.rs#L371-L382, node.rs#L551-L576, sigverify.rs#L102-L110, sigverify.rs#L141-L153, sigverify.rs#L226-L231.

Description: A Write or Aio node starts a pool of signature verification workers between RPC ingress and the deduplication stage. The node accepts `sigverify_workers` as an unrestricted `usize` from `PRIVATE_CHANNEL_SIGVERIFY_WORKERS`. Its startup validation rejects several other zero settings but passes this value unchanged to `start_sigverify_workerpool`.

With a value of zero, the loop over `0..num_workers` spawns no task and the function returns an empty handle list. Returning also drops the only ingress receiver and the only sender to Dedup. RPC admission consequently refuses every transaction as a closed queue. Dedup observes its input close, exits and triggers the node's worker supervision. The binary shuts down the remaining services and exits with an error.

A trusted deployer or its configuration management must set the value to zero. The default is four and the shipped deployment template uses 32. While the zero setting remains, the Write node cannot accept private channel transactions. An Aio node also loses its read service when process supervision shuts it down.

Recommendation: Represent `sigverify_workers` as a positive integer and reject zero in `run_node` before acquiring the writer lease or starting any service. Pass the positive type into `SigverifyArgs` so another caller cannot create an empty pool.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `run_node` rejects zero `sigverify_workers` in Write and Aio modes before acquiring the writer lease or creating workers. Positive counts continue to `start_sigverify_workerpool`. Read mode remains unaffected because it does not create this pool.

3.3.15 Reconciliation script can certify an insolvent escrow

Severity: Low Risk.

Context: reconcile-escrow-balance.sh#L69.

Description: The manual reconciliation script claims that exit code zero means the on-chain escrow balance matches the database's expected balance. Its SQL derives that expectation only from rows whose local operator status is `completed`. That status is not the boundary at which custody changes.

A successful source deposit transfers tokens into the escrow before its indexer row is inserted. The row can remain `pending`, `processing`, `manual_review` or `failed` while those tokens are still held in custody. Conversely, a release changes custody when `ReleaseFunds` lands, and the durable `observed_releases` record is the authoritative accounting evidence even if the withdrawal's asynchronous operator status has not reached `completed`. The production reconciliation query reflects those rules by counting all source deposits through the compared slot and joining observed releases; the shell helper ignores both.

This permits a false PASS during exactly the incident the script is meant to diagnose. For example:

1. One completed deposit of 100 units and one manual-review deposit of 100 units have both transferred into escrow.
2. The escrow should hold 200 units, but a defect or unauthorized payout removes 100, leaving 100 on chain.
3. The script excludes the manual-review deposit, computes `DB_EXPECTED=100`, compares it with the depleted on-chain balance of 100 and prints `PASS — Balances reconcile.`

The omitted depositor is still missing 100 units of backing. An operator or automation relying on the documented exit code can close an incident, continue processing or skip recovery while the custody deficit remains. The continuous Rust reconciliation worker is not a substitute for this tool's truthfulness: this script is explicitly shipped as an independent operational check and may be used when the worker itself is under investigation.

Recommendation: Remove the duplicate accounting formula from the shell script. Call the same snapshot-pinned reconciliation implementation or expose a read-only command that uses every finalized deposit and observed release through one captured slot. Make incomplete indexer coverage a distinct nonzero result rather than treating local terminal status as chain truth. Add a regression with a nonterminal deposit and a landed release whose operator status trails.

Solana Foundation: Fixed in PR PR 388, commit 25235573. The response states that the shell script was deleted and replaced by a read-only indexer reconciliation command that uses the runtime ledger accounting, waits for the escrow indexer to cover the custody slot, and returns distinct mismatch, error, and incomplete-ledger exit codes.

Cantina: Fix verified. `run_reconcile` invokes `run_manual_reconciliation`, which holds the shared live-state lock, rejects an unfinished resync, captures finalized custody, and waits for the escrow checkpoint to cover that slot before calling `reconcile_against_snapshot`. Its ledger query counts deposits in every operator status through the snapshot slot and subtracts only releases recorded in `observed_releases`; coverage, RPC, lock, and database failures cannot produce exit code zero. This restores the invariant that a pass compares slot-aligned custody with all indexed liabilities, independent of operator status, subject only to the configured reconciliation tolerance.

3.3.16 Zero operator batch size stalls all bridge work

Severity: Low Risk.

Context: indexer.rs#L87-L106, indexer.rs#L440-L444.

Description: The operator's batch size controls whether any pending bridge row can enter processing, but its configuration accepts the full `u16` domain and performs no positive-value validation. A value of zero is copied directly into `OperatorConfig` and supplied as the PostgreSQL query's `LIMIT`.

`LIMIT 0` succeeds and returns an empty vector. The fetcher therefore continues its normal polling loop without locking or forwarding a row. It still counts the pending backlog and the process keeps all workers and listeners running, so this is a non-progressing deployment rather than an explicit startup failure.

For the escrow operator, source deposits have already transferred user tokens into custody, but every row remains `pending` and no private receipt is minted. For the withdraw operator, the private burn has already happened, but no Solana release is built. All later rows are affected as well. Nothing automatically repairs the condition; the deployer must diagnose the configuration, set a positive batch size and restart the operator. The pending rows can then be processed, so the outage is recoverable rather than permanent.

Recommendation: Validate `operator.batch_size > 0` before opening the database, acquiring the live-state lock or starting workers. Centralize semantic validation for all operator counts, durations and capacities, and add a startup test for every zero boundary. Treat an unexpected empty claim while a positive pending count exists as a health failure with the configured limit in the diagnostic.

Solana Foundation: Fixed in PR 375, merged as `464ca71d27c9b64830642febe419855f64f061c0` (implementation commit `74bc1c818a73f7d7fd798a0d1181cdc1b4cf16dd`). `validate_operator_startup` now rejects a zero `operator.batch_size`; `build_operator_config` invokes it before `PostgresDb::new`, and `operator::run` repeats the check before the live lock, storage access or worker startup.

Cantina: Fix verified. Both operator roles must pass the shared positive-batch-size check before the fetcher can call `get_and_lock_pending_transactions`, so `LIMIT 0` is unreachable through the supported operator startup paths. A zero value now fails startup before any bridge row can be claimed, restoring the invariant that every running deposit or withdrawal operator has a positive work-claim limit.

3.3.17 Missing webhook suppresses insolvency alerts

Severity: Low Risk.

Context: indexer.rs#L88-L105, indexer.rs#L440-L462, reconciliation.rs#L675-L711, reconciliation.rs#L846-L872.

Description: The escrow operator documents `reconciliation_webhook_url` as required because a confirmed custody shortfall or over issuance needs an operator response. Its configuration schema instead defaults the field to `None`. Startup copies that value into `OperatorConfig` and validates only signer configuration before starting the operator.

When reconciliation later confirms an insolvency, it still sets the durable halt, quarantines withdrawals and forces `/health` to return 503. It then passes the absent URL to `send_webhook_alert()`. That function logs a warning and returns success without sending the mismatch. The runbook describes this webhook as the halt's only dedicated alert. The health failure is a secondary service outage signal.

A trusted deployer who omits the field therefore starts an apparently valid escrow operator that cannot deliver its promised insolvency alert. The bridge remains fail closed, but response and manual recovery can be delayed until someone notices the unhealthy service or reviews its logs. The example and packaged escrow configurations include a value, so this requires a custom or incompletely rendered deployment.

Recommendation: Reject escrow operator startup when `reconciliation_webhook_url` is absent or blank. Apply the check before opening storage or starting workers, while continuing to ignore the setting for the Withdraw role. Keep the existing durable halt and health failure as independent fallback signals.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `validate_operator_startup` rejects an absent or blank reconciliation webhook for Escrow before the live lock or schema initialization. The binary validates before opening storage and passes the trimmed URL to reconciliation. Withdraw remains allowed to omit it. Endpoint delivery remains an operational requirement.

3.3.18 Invalid operator signer silently uses admin

Severity: Low Risk.

Context: config.rs#L418-L429, signer_util.rs#L107-L116, signer_util.rs#L265-L269.

Description: The operator configuration permits the admin signer to serve as the operator only when `OPERATOR_SIGNER` is not set. However, `OPERATOR_SIGNER_INSTANCE` converts every error from `load_signer(SignerRole::Operator)` into `None`. This includes a selected backend with a missing credential, an invalid key or an unsupported signer type. `operator_signer()` then replaces the configured operator with `ADMIN_SIGNER_INSTANCE`. `OperatorConfig::validate_signers()` returns success.

This affects a deployment that registered a distinct operator for its escrow instance but supplied an invalid operator backend configuration. `ProcessorState` derives the submitted operator PDA from the substituted admin key. `ReleaseFunds` and bitmap rotation therefore cannot use the PDA that authorizes the registered operator. The process appears to start successfully, but withdrawals and

rotation remain unavailable until a trusted deployer corrects the configuration and restarts it. The shipped Compose configuration aliases the two keys, so this failure requires a deployment that uses the separately documented operator identity.

Recommendation: Use the admin fallback only when `OPERATOR_SIGNER` is absent. Retain and propagate every error that occurs after this variable is present so `OperatorConfig::validate_signers()` rejects the configuration before any operator worker starts.

Fix review: **Solana Foundation**: Fixed in commit 471e2e4 (PR PR 382).

Cantina: Fix verified. `load_operator_signer` now returns the administrator fallback only when `OPERATOR_SIGNER` is absent and propagates every error from a present configuration; `init_operator_signer` makes such an error fatal when `validate_signers` forces the operator key before storage or workers start. Releases and bitmap rotations still use `operator_signer`, so an invalid distinct operator can no longer be silently replaced by the administrator, while the documented fallback remains available for an intentionally absent setting.

3.3.19 Weak JWT secrets permit session forgery

Severity: Low Risk.

Context: config.rs#L17-L25, config.rs#L108-L130, lib.rs#L101-L110, lib.rs#L416-L420, lib.rs#L524-L527.

Description: `JWT_SECRET` is the shared HS256 key that limits issued tokens to their authenticated subject and 24 hour lifetime. `Config::validate()` does not inspect this value. The gateway's `configured_secret()` likewise accepts every nonblank string and constructs a decoding key directly from its bytes.

A deployment can therefore enable authentication with a trivially weak value such as one character. Any registered user receives a valid token for its own known UUID and can test candidate HMAC keys against that token without contacting the service. After recovering the shared key, the user can sign a replacement token with an arbitrary expiry. The auth service and public gateway accept the forged token because its signature, issuer and audience are valid.

This defeats the configured 24 hour session boundary and permits impersonation of another subject whose UUID becomes known. The gateway confirms an asserted Operator role against PostgreSQL, which limits this path to session forgery and known subject impersonation. The deployment must supply a weak secret despite the repository guidance to generate a random value of at least 32 bytes.

Recommendation: Reject an enabled `JWT_SECRET` shorter than 32 bytes before either service connects to its database or begins listening. Use the same validation rule in auth and gateway while preserving the exact accepted bytes as the shared HMAC key.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. Auth `Config::validate` and gateway `resolve_jwt_secret` reject enabled secrets shorter than 32 trimmed bytes before database access. Accepted values retain their exact HMAC bytes and errors omit the secret. Existing short keys require coordinated rotation. Length enforcement does not establish entropy.

3.3.20 Invalid mint options initialize private mints

Severity: Low Risk.

Context: admin.rs#L323-L333.

Description: Private channel administrator transactions execute `InitializeMint` through `AdminVm` instead of the SPL Token program. This path creates persistent receipt mint state and is intended to preserve the supported SPL instruction semantics.

`process_initialize_mint()` treats the freeze authority byte as `Some` only when it equals `1`. Every other value becomes `None`. The SPL Token decoder accepts only `0` and `1`, so a tag such as `2`

must reject the instruction. `AdminVm` instead creates the mint, returns success and exposes the initialized account to normal settlement with no freeze authority.

This affects an administrator signed initialization produced by a defective or custom builder. The first malformed attempt permanently initializes the mint under semantics that standard SPL Token execution rejects. A corrected retry then fails because the account is already initialized, so recovery requires explicit state repair or mint migration. The configured administrator signature is still required, which limits this to a low likelihood integration and recovery hazard rather than an unprivileged privilege escalation.

Recommendation: Decode the complete instruction with `TokenInstruction::unpack()` before constructing the mint. Alternatively, explicitly accept freeze authority tags `0` and `1` and return `InvalidInstructionData` for every other value. Only return a successful admin result after all fields match SPL Token instruction semantics.

Solana Foundation: Fixed in `471e2e4a` (PR PR 382). `process_initialize_mint()` now accepts freeze-authority tag `0` as absent and tag `1` as present, requires the 32-byte authority for tag `1`, and returns `InvalidInstructionData` for every other tag before resolving or constructing the mint account.

Cantina: Fix verified. Commit `471e2e4ac1d4263f3e061143792d149ff3264eab` routes both `InitializeMint` variants through the strict tag match in `AdminVm::process_initialize_mint()`, and a rejected instruction returns no account changes. This restores the invariant that only canonical SPL option encodings can initialize a private mint, so malformed tags cannot persist a mint or consume its initialization state.

3.3.21 Startup supply check ignores bps tolerance

Severity: Low Risk.

Context: reconciliation.rs#L55-L60, reconciliation.rs#L317-L376.

Description: Startup reconciliation is meant to apply the configured absolute and relative allowances before it decides that escrow custody cannot cover the channel supply. `startup_tolerance_raw()` returns the larger of `mismatch_threshold_raw` and `reconciliation_tolerance_bps` applied to custody. The ledger liability arm uses this result, but `measure_supply_breaches()` compares `supply - custody` only with `mismatch_threshold_raw`.

For example, consider custody of 1,000 raw units, channel supply of 1,005, an absolute threshold of zero and a percentage tolerance of 100 bps. The effective configured tolerance is ten units, so the five unit gap should be accepted. The supply arm nevertheless treats the gap as a breach because it compares five with the zero absolute threshold. After the same gap appears in three readings, `check_channel_supply_invariant()` returns `SupplyExceedsCustody`.

This affects deployments that configure a nonzero percentage tolerance and encounter a small standing or observation gap inside that allowance. The escrow indexer refuses to start, leaving subsequent deposits and release observations unindexed until an operator changes the configuration or state and restarts the process.

Recommendation: Compare the supply gap with `startup_tolerance_raw(config, on_chain_custody)` and include that effective tolerance in the supply alert and error details. Preserve the existing repeated reading rule and the strict behavior when custody is zero.

Solana Foundation: Fixed in PR PR 388, commit `2523557362a004fd75d7f62f1bfbd0c933e6908b`. The Foundation states that the startup supply check now uses the same effective raw-or-bps tolerance as the ledger check, reports that tolerance, and retains the three-reading and zero-custody rules.

Cantina: Fix verified. `measure_supply_breaches` now compares `supply - custody` with `startup_tolerance_raw(config, on_chain_custody)`, which takes the larger of `mismatch_threshold_raw` and the overflow-safe custody-based bps allowance. `check_channel_supply_invariant` still requires three confirming readings and now logs the effective tolerance; zero custody produces a zero

bps allowance. This restores the invariant that startup accepts gaps at or below the configured combined tolerance and rejects persistent gaps above it, with no material bypass found.

3.3.22 Zero auth database pool crashes both services

Severity: Low Risk.

Context: config.rs#L34-L36, config.rs#L110-L130, main.rs#L26-L38, lib.rs#L112-L117, lib.rs#L1493-L1541.

Description: The authentication service and the gateway consume the same documented `AUTH_DATABASE_MAX_CONNECTIONS` deployment setting. Both command line definitions accept it as an unrestricted `u32`. `Config::validate()` in auth checks other configuration relationships but does not reject zero and gateway startup performs no check for this field.

Each service passes the value to `PgPoolOptions::max_connections()` and calls `connect()`. SQLx constructs its pool with a fixed capacity `ArrayQueue` at that point. A capacity of zero reaches `ArrayQueue::new(0)`, which asserts that the capacity must be nonzero and panics before either service binds its public listener.

The packaged Compose deployment passes this one environment value to both services and restarts them when they exit. A trusted deployment mistake that sets it to zero therefore puts both the authentication service and the public JSON RPC gateway into restart loops. Users cannot register or log in and no public RPC request can reach the private channel until an operator corrects the value and restarts the services. The shipped value is ten and an untrusted caller cannot change it, which limits the severity.

Recommendation: Represent both command line fields as `NonZeroU32` and pass the inner value to `PgPoolOptions::max_connections()`. This makes Clap reject zero as a configuration error before either service starts.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. Auth and gateway now parse the shared database pool setting as `NonZeroU32`. Both pool builders use its positive inner value, so zero is rejected before SQLx can construct either pool.

3.3.23 Zero sample period stops the write node

Severity: Low Risk.

Context: node.rs#L138-L144, node.rs#L214-L238, node.rs#L210-L237, node.rs#L433-L449, node.rs#L551-L570, settle.rs#L715-L719, settle.rs#L1035-L1057.

Description: The node periodically stores performance samples from its settler. `PRIVATE_CHANNEL_PERF_SAMPLE_PERIOD_SECS` controls this interval and the command line parser accepts it as an unrestricted `u64`. `run_node()` rejects several other zero durations and capacities but does not validate this field before starting a Write or Aio node.

The node passes the value unchanged to `start_settle_worker()`. The worker supplies it to `tokio::time::interval_at()` as the interval period. Tokio panics when that period is zero. The worker wrapper handles an `Err` returned by its inner future but it does not catch a panic, so the spawned settler task terminates.

`NodeHandles::wait_for_any_worker_quit()` treats any completed worker as an unexpected exit. The binary consequently shuts down all remaining services and returns an error. A trusted deployment mistake that sets the sampling period to zero therefore prevents the Write node from processing private channel transactions. An Aio node also loses its read service. The shipped default is 60 seconds and no remote caller can change the setting, which limits the severity.

Recommendation: Represent `perf_sample_period_secs` as `NonZeroU64` and reject zero before `run_node()` acquires the writer lease or starts any service. Pass the positive value into `SettleArgs` so every caller preserves the interval constructor's requirement.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `run_node` requires `perf_sample_period_secs` to be between 1 and 65,535 in Write and Aio modes before acquiring the writer lease. The value passed to the settler is therefore a valid Tokio interval and fits the stored `u16` period.

3.3.24 Zero channel capacity crashes the operator

Severity: Low Risk.

Context: indexer.rs#L87-L105, indexer.rs#L440-L463, config.rs#L359-L369, config.rs#L418-L429, operator.rs#L23-L33, operator.rs#L53-L65, operator.rs#L128-L130.

Description: Both operator roles use `channel_buffer_size` for the bounded queues between their fetcher, processor and sender workers. The operator configuration accepts this field as an unrestricted `usize` and startup validates only signer configuration before passing the value to `operator::run()`.

`operator::run()` acquires the live database lock and initializes the schema, then supplies the configured value to two `tokio::sync::mpsc::channel()` calls. Tokio requires bounded channels to have a positive capacity and panics when the capacity is zero. Neither queue nor any bridge worker is therefore created under this setting.

A trusted deployment mistake that sets the capacity to zero terminates the operator during every startup attempt. The Escrow role cannot mint indexed deposits and the Withdraw role cannot release indexed burns until the value is corrected and the process restarts. Every shipped configuration uses a positive capacity and no remote caller can alter the field, which limits the severity.

Recommendation: Represent `channel_buffer_size` as `NonZeroUsize` in the deserialized and runtime operator configurations. Pass its inner value to both queue constructors. Alternatively, reject zero at the beginning of `operator::run()` before acquiring the live lock or initializing storage.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `validate_operator_startup` rejects a zero channel buffer in both roles. The binary checks before opening storage and `operator::run` checks before acquiring the live lock or creating either bounded queue. The accepted residual for extremely large capacities remains.

3.3.25 Zero connection limit disables core RPC

Severity: Low Risk.

Context: node.rs#L45-L47, node.rs#L197-L203, node.rs#L210-L237, node.rs#L525-L533, server.rs#L39-L75.

Description: The core node uses `max_connections` to bound simultaneous RPC clients without disabling its RPC interface. The command line accepts `PRIVATE_CHANNEL_MAX_CONNECTIONS` as an unrestricted `usize`. `run_node()` rejects several other zero values but passes this field unchanged into `start_rpc_service()`.

At zero the service binds its port and logs that it is listening. It then constructs a semaphore with no permits. Every accepted socket fails `try_acquire_owned()` and is dropped before HTTP handling. The process remains alive. Health requests fail. Ledger reads and transaction submissions also fail.

A trusted deployment mistake can therefore leave any core node running with its complete RPC interface unavailable until the value is corrected and the process is restarted. The shipped settings are positive. A remote caller cannot change this field. These facts limit the impact.

Recommendation: Represent `max_connections` as `NonZeroUsize` in the command line configuration. Carry the positive type through `NodeConfig` and `RpcServiceConfig`. Reject zero before database setup or service startup so every running core listener can admit at least one connection.

Solana Foundation: Fixed in commit 464ca71.

Cantina: Fix verified. `run_node` enforces a connection limit from 1 through 10,000,000 in Read, Write and Aio modes before database or listener startup. `start_rpc_service` receives this checked value, preventing a configured node from binding an RPC listener with no admission permits.

3.3.26 Username throttling does not limit credential guesses

Severity: Low Risk.

Context: login.rs#L31-L59.

Description: The authentication service documents `AUTH_USERNAME_ATTEMPTS_PER_MINUTE` as a limit on credential attempts against one username across all source IPs. This is intended to stop a distributed password-guessing campaign from bypassing the per-IP limiter.

The login handler does not consult that bucket before checking a credential. It first queries the user, performs the real or dummy Argon2 verification, and calls `charge_failed_attempt()` only after verification has failed. Once the username bucket is exhausted, later wrong guesses return `429`, but every guess has still reached the database and Argon2. More importantly, a correct guess bypasses the exhausted bucket and immediately receives a token.

An attacker distributed across enough IP addresses can therefore continue testing passwords against the same account without any per-account ceiling. The limiter changes the status code of failed guesses but does not limit the number of guesses or the expensive work they cause. The Argon2 semaphore bounds simultaneous work and the per-IP limiter constrains each source, but neither provides the documented cross-IP attempt limit.

The current ordering deliberately prevents an unauthenticated party from hard-locking another user's account. That availability goal is reasonable, but it means this mechanism cannot also be described or relied upon as a password-guessing boundary.

Recommendation: Apply a real account-level friction mechanism after repeated failures, such as progressive delays, an additional challenge, MFA, or a risk-based check that a correct password alone does not bypass until the cooldown expires. Avoid a simple attacker-drainable hard lockout. Keep the dummy-hash path and uniform error behavior so the change does not reintroduce username enumeration. If the current behavior is retained intentionally, rename and document the setting as failed-response shaping rather than an attempt limit, and do not treat it as brute-force protection.

Solana Foundation: Addressed in commit 471e2e4a (PR PR 382) by retaining the behavior, renaming the setting to `AUTH_USERNAME_FAILED_LOGINS_PER_MINUTE`, and documenting that it shapes failed login and taken-name registration responses while guesses across many IPs remain unbounded.

Cantina: Fix verified. The renamed setting and its documentation now match the retained implementation: only a failed login or taken-name registration charges the username bucket after database and Argon2 work, while a valid password reaches token issuance without consulting that bucket. The invariant is therefore explicit that this control only changes exhausted failure responses to `429`; it is not a per-account bound on guesses or hashing work, which remain governed by the per-IP limiter and Argon2 concurrency cap.

3.3.27 Unbounded pending-remint recovery can stall withdrawal startup

Severity: Low Risk.

Context: db.rs#L1250-L1254.

Description: Pending compensation remints are durable so a restart cannot forget a failed withdrawal. During sender startup, `get_pending_remint_transactions_internal()` selects every withdrawal in `PendingRemint` status with `fetch_all()` and no limit. `recover_pending_remints()` then parses

every row and every stored signature into an unbounded in-memory `Vec<PendingRemint>` before the sender event loop begins.

The runtime path has the same all-at-once behavior. Every 500 milliseconds, `process_pending_reminits()` drains the complete vector, separates all matured entries, and processes each matured entry sequentially. Classification can perform several destination RPC, bitmap, release-record, source RPC, and database operations before advancing to the next entry.

A prolonged set of ambiguous or journaled remint attempts can leave many rows durably pending. Failing release destinations can also create a burst approaching the sender's in-flight capacity before the first 32-second finality deadline matures. A restart at that point allocates and expands the entire durable backlog before it can service fresh work. After startup, one matured batch can monopolize the single sender loop for an unbounded period. A sufficiently large backlog can exhaust memory or keep new withdrawals, confirmations, and bitmap rotation from being processed.

The separate `MAX_IN_FLIGHT` semaphore does not bound this recovery collection: it controls active fire-and-forget submissions, while pending-remint entries relinquish that submission slot and are stored in their own unbounded vector.

Recommendation: Keep pending remints as a database-backed work queue. Fetch or claim a bounded number ordered by deadline and transaction id, using a lease or `FOR UPDATE SKIP LOCKED` if multiple workers are possible. Limit both the number hydrated at startup and the number processed per sender tick, then yield to confirmations and new submissions before claiming another batch. Add metrics and readiness thresholds for the durable pending-remint backlog rather than mirroring the complete backlog into memory.

Solana Foundation: Fixed in PR 384 and PR 389.

Cantina Managed: Verified fix.

3.3.28 Invalid CORS configuration crashes gateway requests

Severity: Low Risk.

Context: lib.rs#L97-L99.

Description: `GATEWAY_CORS_ALLOWED_ORIGIN` is accepted as an unrestricted string and stored unchanged in `Gateway`. Startup logs the value and binds both listeners without parsing it as an HTTP header value.

Every response path later inserts the string into `Access-Control-Allow-Origin`. Locally constructed responses use `Response::builder().header(...).body(...).unwrap()`, while forwarded responses call `HeaderValue::from_str(...).unwrap()` directly. Header values containing a newline, carriage return, or another invalid byte therefore panic the connection task instead of returning a controlled error.

Because the listener is already running, the process can appear started while ordinary RPC, OPTIONS, `/health`, and `/ready` requests terminate their spawned connection tasks. Container health checks fail and clients see dropped connections until the configuration is corrected and the service restarted. The value is controlled by a trusted deployer rather than a remote request, which limits the severity.

Recommendation: Parse `GATEWAY_CORS_ALLOWED_ORIGIN` into `HeaderValue` once during startup and return a descriptive configuration error if it is invalid. Store the typed value in `Gateway` and clone it into response headers without any `unwrap()`. Consider validating that the configured value is either `*`, `null`, or a syntactically valid single origin, rather than accepting arbitrary valid header text.

Solana Foundation: Fixed in commit 471e2e4a (PR PR 382). `run` now parses `GATEWAY_CORS_ALLOWED_ORIGIN` as a `HeaderValue` before logging the value, connecting to the auth database, or binding either gateway listener, and returns a configuration error when parsing fails.

Cantina: Fix verified. The shipped gateway binary parses its arguments and enters `run`, which calls `check_cors_origin` before moving the unchanged origin into `Gateway::new`; no production caller

constructs or serves a `Gateway` outside this validated path. Because startup and the remaining response insertion use the same `HeaderValue` conversion, an accepted origin can no longer trigger the downstream `unwrap`. This restores the invariant that an invalid configured CORS header fails startup before request serving begins.

3.3.29 Zero retention silently deletes every successful backup

Severity: Low Risk.

Context: pg-backup.sh#L26-L29.

Description: The PostgreSQL backup sidecar accepts `PG_BACKUP_RETENTION_COUNT` as an unchecked shell string. The documented default is three, but no validation requires the supplied value to be a positive integer.

When retention is set to zero, every successful backup is immediately eligible for deletion. `prune_old_backups()` calculates `to_remove=count-RETENTION`, so after a successful `pg_basebackup` it selects and recursively removes every `base_*` directory, including the backup that just completed. `prune_old_wal()` then finds no retained backup and returns without establishing a WAL recovery floor.

The sidecar continues running and prints `Backup complete` on every interval, which can make monitoring report apparent success even though no restore point exists. A later database loss or destructive resync failure is therefore unrecoverable from the configured backup volume. Only a trusted deployment mistake can set this environment value, which limits the severity.

Recommendation: Validate `PG_BACKUP_RETENTION_COUNT` and `PG_BACKUP_INTERVAL_HOURS` before waiting for PostgreSQL. Require canonical positive integers and refuse zero. After pruning, verify that at least one complete base backup remains and expose that state through a health check or metric. Avoid logging a backup cycle as successful until retention validation and post-prune verification have both passed.

Solana Foundation: Fixed by PR 375 and PR 383.

Cantina Managed: Verified fix.

3.3.30 Zero operator intervals hammer dependencies and defeat reconciliation debounce

Severity: Low Risk.

Context: indexer.rs#L87-L106.

Description: The operator configuration accepts the database poll, reconciliation, fee-payer monitoring, and confirmation polling intervals as unrestricted `u64` values. Startup converts them directly into zero-length `Duration` values and validates only signer configuration.

A zero database poll interval makes the fetcher continuously count and lock pending rows with no pacing when the queue is empty. A zero fee-payer interval continuously calls `getBalance`, and a zero confirmation interval removes the delay between repeated signature-status requests. These loops remain bounded by individual database or RPC latency, but they run at the maximum rate those dependencies can sustain and can consume the same capacity needed for settlement.

A zero reconciliation interval also changes a safety property. The insolvency detector halts only after `HALT_CONFIRM_TICKS` consecutive observations so a transient or stale cross-system view does not immediately freeze both bridge directions. With no delay, the worker can take all confirming observations back-to-back against effectively the same finalized ledger and replica state, converting the debounce into repeated reads of one snapshot and tripping the durable global halt almost immediately.

A trusted configuration mistake can therefore overload PostgreSQL and both RPC services, delay legitimate bridge work, and cause a transient mismatch to be treated as a confirmed insolvency. No remote caller can change these settings, which limits the severity.

Recommendation: Represent all operational intervals as validated positive duration types and reject zero before acquiring locks or spawning workers. Set meaningful minimums for reconciliation and fee-payer polling, not merely nonzero values. Define reconciliation confirmation in terms of distinct finalized slots or elapsed time in addition to tick count, so an accidentally aggressive interval cannot sample the same state repeatedly and satisfy the halt threshold.

Solana Foundation: Fixed in PR 375, commit `464ca71`. The shared startup validator rejects zero database-poll, retry, reconciliation, fee-payer, and confirmation intervals, while the shipped binary requires at least 10 seconds for reconciliation and 5 seconds for fee-payer polling. The finalized-read and counter-hold behavior used by reconciliation was established in PR 334, commit `64946e5`.

Cantina: Fix verified. TOML and `OPERATOR_*` values converge on `build_operator_config`, and `operator::run` repeats the positive-value checks for direct library callers before taking a lock or starting workers. `run_reconciliation` completes one check and then sleeps, so the binary's 10-second floor places at least 20 seconds between the first and third breach observations. Custody reads are bound to a recent finalized anchor; stale or unreadable inputs fail the tick and hold the permint breach counters. This restores the invariant that supported operator configuration cannot create zero-delay dependency loops or collapse the insolvency debounce into back-to-back observations.

3.3.31 Critical task exits panic operator shutdown

Severity: Low Risk.

Context: operator.rs#L364-L435, operator.rs#L457-L474, shutdown_utils.rs#L464-L482, shutdown_utils.rs#L502-L533.

Description: The operator supervises its fetcher, processor, sender, storage writer and recovery worker so an unexpected worker exit stops the role and drains the remaining pipeline. `operator::run` polls each worker's `JoinHandle` by mutable reference inside `tokio::select!`. When a worker branch wins, Tokio has already returned that handle's output and marks the handle as consumed.

The same handle is then moved into `shutdown_operator` with every worker that is still running. The shutdown stages await the completed worker again. Tokio rejects that second poll with `JoinHandle polled after completion`, contrary to the source comment that describes it as a no op.

Any dependency, channel or internal error that makes a monitored worker return therefore sends the operator through a panic instead of the intended graceful shutdown. The process terminates before the remaining processor, sender, recovery and storage writer stages are deliberately drained. Buffered bridge work must wait for crash recovery after restart even though the code selected the graceful cleanup path.

Recommendation: Retain the identity and result of the worker that completed during supervision. Pass only the remaining handles into `shutdown_operator`, then report the saved worker result after those handles have drained. Do not poll the supervision winner a second time.

Solana Foundation: Fixed in PR 388, commit `25235573`.

Cantina: Fix verified. `operator::run` now retains the identity and join result of all seven supervised workers, replaces only the consumed winner with an awaitable stand-in, and drains the remaining workers. Join failures are logged before the lost-lock and shutdown-error returns; live-lock loss aborts without draining, while a successful drain returns `CriticalTaskExited`, which propagates through the indexer binary as a nonzero process exit. This restores the invariant that an unexpected supervised-worker exit cannot re-poll its `JoinHandle` or be reported as a successful operator stop.

3.3.32 Concurrent schema setup aborts an indexer

Severity: Low Risk.

Context: db.rs#L281-L633.

Description: `PostgresDb::init_schema` runs its trigger migration as three separate pool operations. Each caller creates or replaces `assign_withdrawal_nonce`, drops `trigger_assign_withdrawal_nonce` and then creates that trigger again. The Escrow indexer, Withdraw indexer and both operators share one PostgreSQL database and can initialize this schema at the same time during an ordinary deployment.

Two callers can both finish the drop before either caller creates the trigger. The first create succeeds and the second returns `trigger "trigger_assign_withdrawal_nonce" for relation "transactions" already exists`. This error escapes schema initialization and terminates the affected indexer. The remaining role can continue running while one bridge direction stops ingesting events until that process starts again successfully.

The complete PostgreSQL integration flow reproduced this ordering while starting the source and destination indexers together. One indexer exited with the duplicate trigger error and the expected withdrawal never appeared. A focused test with 32 simultaneous production `init_schema` calls reproduced the same error. Wrapping only the trigger creation in a duplicate object handler made the same complete PostgreSQL and Redis workflows pass, which isolates this race from the instrumentation delays that exposed it.

Recommendation: Serialize the complete schema migration with one PostgreSQL advisory lock held on one connection. Prefer a versioned migration runner that records each migration exactly once. Make every remaining data definition step safe under concurrent startup and add a regression that initializes every supported service against one fresh database at the same time.

Solana Foundation: Fixed in PR 329 and PR 335.

Cantina Managed: Verified fix.

3.3.33 Yellowstone builds require the RPC feature

Severity: Low Risk.

Context: source.rs#L25-L26.

Description: The indexer declares `datasource-rpc` and `datasource-yellowstone` as independent optional features. Its configuration validation also has a dedicated Yellowstone branch that reports the Yellowstone feature as the required build capability. A binary with only that feature should therefore compile without the RPC polling implementation.

The common Escrow and Withdraw parsers import instruction types from the RPC polling module. The Yellowstone source imports another RPC polling helper and the same RPC instruction types. The main indexer module gates its `Duration` import on the RPC feature even though shutdown code uses that type for every datasource.

`cargo check -p private-channel-indexer --locked --no-default-features --features datasource-yellowstone` consequently fails with five compiler errors. The equivalent RPC only test profile passes 1,031 library tests. An operator cannot build the declared Yellowstone profile without also enabling the RPC implementation, so the intended isolated deployment and its feature specific checks are unavailable.

Recommendation: Move the instruction metadata types and shared target predicate into the common datasource module. Import `Duration` wherever shutdown uses it and gate only logic that truly needs RPC polling. Add locked checks for the default profile, RPC alone and Yellowstone alone to continuous integration.

Solana Foundation: Fixed in PR 382, commit `471e2e4a`.

Cantina: Fix verified as an accepted redesign. The shared instruction types and target predicate now live in the common datasource modules, and `Duration` is available to every shutdown path. The independent Yellowstone-only profile was retired rather than restored: `datasource-yellowstone` now enables `datasource-rpc`, matching the RPC-backed startup anchor and gap repair required by the Yellowstone runtime. Normal Cargo feature resolution cannot select Yellowstone without RPC, and CI checks the resulting default dual-datasource profile plus RPC alone.